WeCreat Lumos Ultra — LightBurn Configuration Project

Banner artwork is aspirational — it shows where this project is going, not where it is. For actual, current status of every feature see the feature inventory. Lens part numbers shown are decorative.

An open, evidence-graded effort to build a working **LightBurn** device configuration for the **WeCreat Lumos Ultra** (6 W UV + optional 60 W / 100 W JPT MOPA fiber), covering every field lens, work mode and accessory.

WeCreat does not publish a LightBurn profile for the Lumos Ultra. They publish one for the Vision, Vision Pro, Vista and Lumos — but not the Ultra — while simultaneously listing "Lightburn" as supported software on the Ultra product page. This repository exists to close that gap in the open, with contributions from anyone who owns the machine.

Status: Stages 0 and 1 complete on hardware. Architecture, USB identity, baud rate and firmware behaviour are **confirmed against a physical Lumos Ultra**. The device profiles in `profiles/draft/` remain untested. Every claim carries an evidence grade.
Read SAFETY.md before you connect anything.

⚠ If you own a Lumos Ultra, do not just import WeCreat's Lumos profile

The Ultra identifies itself over USB as `[WeCreat Lumos :ver 000240]` — it does **not** say "Ultra". Nothing in the handshake distinguishes the two machines, so WeCreat's official `WeCreat-Lumos-v1.5.1bdev` **will connect to an Ultra without complaint**.

That profile declares a **116 × 116 mm** workspace. The Ultra's field is **210 × 210 mm**. Jobs would be silently mis-scaled and mis-placed, with no error and no warning. (evidence)

What we already know (and how well)

Finding	Grade
WeCreat's LightBurn profiles are **GRBL / G-code serial devices**, not galvo controllers	**CONFIRMED** — read out of WeCreat's own `WeCreat-Lumos-v1.5.1bdev`: `"Name": "GRBL"`, `"Type": "Serial"`, `BaudRate 1000000`
The Lumos-family controller is a **Linux SBC front-end with a separate MCU**	**CONFIRMED** — WeCreat's bundled RNDIS driver binds `USB\VID_0525&PID_a4a2` and `USB\VID_1d6b&PID_0104&MI_00` (Linux USB gadget IDs); the reverse-engineered REST endpoint is literally `/test/cmd/mcu`
WeCreat uses a **private M-code dialect** that is renumbered per machine model	**CONFIRMED** — see docs/04-mcode-dictionary.md
MakeIt! 3.0.6 ships **nine distinct Lumos Ultra work modes**	**CONFIRMED** — enumerated from the shipping application bundle, see docs/06-modes-and-lenses.md
The Ultra specifically is a GRBL-over-serial device	**CONFIRMED-hardware, 2026-08-24** — a physical Lumos Ultra enumerates as a CH340 serial bridge (`VID_1A86&PID_7523`) plus a Linux RNDIS gadget (`VID_0525&PID_A4A2`), with **no galvo controller present**. Capture · analysis

Finding	Grade
The controller sits on a private USB network at `192.168.42.0/24`	**CONFIRMED-hardware** — the RNDIS adapter comes up at `192.168.42.100`, putting the controller at (almost certainly) `192.168.42.1`
LightBurn's galvo features (Q-Pulse Width, lens bulge/skew correction, galvo rotary, Split Marking, 3D depth-map) are **unavailable** on a GRBL-typed device	**CONFIRMED for the Lumos**, and now that the Ultra is confirmed GRBL, it inherits the same limits

LightBurn's galvo-class **UI fields** are therefore unavailable. But the underlying capability turned out not to be:

✅ MOPA pulse width and frequency ARE controllable

Two real MOPA jobs differing only in pulse width produced G-code differing by **exactly one line** — `M39P200` → `M39P500`.

```
M38F<kHz>     ; MOPA frequency        CONFIRMED-vendor
M39P<ns>      ; MOPA pulse width      CONFIRMED-vendor
M18S0         ; MOPA source select    CONFIRMED-vendor
```

LightBurn's GRBL class supports **per-layer custom G-code** — the exact granularity a MOPA material library needs. You lose the convenient spin-boxes; you keep the control. (evidence)

Still genuinely out of reach through LightBurn: 16-bit depth-map relief and K9 internal 3D engraving, which need the galvo device class itself. See docs/09-k9-and-mopa-limits.md.

What this repository contains

```
docs/         The research. Architecture, feature inventory, M-code dictionary,
              connectivity, modes and lenses, accessories, cameras, K9 crystal,
```

```
                .lbdev file format, MakeIt internals, and a full source list.
profiles/       Draft LightBurn device profiles (.lbdev), one per lens/source/mode.
tools/          Scripts to probe your workstation, fetch vendor reference profiles,
                summarize any .lbdev, generate profiles, and inspect MakeIt.
captures/       Where contributors drop their hardware findings. Templates
included.
reference/      Local-only working area. Gitignored. Vendor files are NOT
redistributed.
```

Quick start (no laser required)

```
# 1. See what your PC has
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\probe-workstation.ps1

# 2. Pull WeCreat's official profiles for the other machines, as reference
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\fetch-vendor-
lbdev.ps1

# 3. Read any .lbdev in human form
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\summarize-lbdev.ps1
.\reference\vendor-lbdev

# 4. Generate the draft Ultra profiles
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\build-profiles.ps1
```

Quick start (with a Lumos Ultra)

Do not skip SAFETY.md. If the laser lives on a different machine from the one you author on, docs/13-test-machine-setup.md covers getting the repo onto it and checking it is ready. Then work through docs/03-bringup-plan.md in order. Stage 0 and Stage 1 involve no beam at all and are the most valuable thing any owner can contribute right now.

The three highest-value contributions today, in order:

1. **USB enumeration.** Plug in the Ultra, run `tools/probe-workstation.ps1`, and open an issue with the output. This settles the architecture question outright.
2. **The serial banner and `$$` dump.** Connect at 1,000,000 baud and capture what the machine says on connect. Expect something shaped like `[WeCreat Lumos Ultra :ver ######]`.

3. **A MakeIt job capture.** Run the same tiny job twice, changing exactly one thing (UV vs MOPA; two pulse widths; two focus heights), and capture the G-code. This is how the Ultra's M-code table gets written.

Templates for all three live in `captures/templates/`.

How to help

See CONTRIBUTING.md. Short version: we grade every claim, we never guess in the documentation, and we never ask anyone to send an unknown M-code to a 100 W fiber laser to find out what it does. Issues are structured by kind — feature status, M-code discovery, profile bug, calibration data.

You do not need to own an Ultra to help. Reviewing the docs, improving the tooling, testing the profile generator, and chasing down sources are all useful.

Scope and honesty

This project can produce a **LightBurn configuration**. It cannot invent controller capabilities that WeCreat has not exposed. Where a feature turns out to be MakeIt-only, we will say so plainly and document *why*, rather than shipping a profile that pretends otherwise.

Legal

WeCreat's own `.lbdev` files, firmware, and application code are **not redistributed here**. `tools/fetch-vendor-lbdev.ps1` downloads them from WeCreat's public CDN to your own machine for comparison, and `reference/` is gitignored. Where this project inspects a locally installed copy of MakeIt!, it does so to determine the interface the operator's own hardware expects — an interoperability purpose — and publishes only the resulting factual findings, never WeCreat's code.

This project is not affiliated with, endorsed by, or supported by WeCreat or LightBurn Software.

01 — Architecture: what the Lumos Ultra actually is, from LightBurn's point of view

Evidence grades used throughout this repository

Grade	Meaning
CONFIRMED-vendor	Stated or demonstrated by WeCreat's own artifacts — a shipped file, a support article, the application bundle
CONFIRMED-community	An owner or reviewer reports the observed behaviour from testing
INFERRED	Deduced from adjacent confirmed facts. Plausible, not proven
UNKNOWN	Present in vendor output, or required by the project, with no established meaning

---

1. The headline

LightBurn talks to WeCreat machines as **GRBL-flavoured G-code over a serial port**. Not as a galvo controller.

This is not a guess. WeCreat's own `WeCreat-Lumos-v1.5.lbdev` — the profile for the Ultra's closest sibling, which is *also* a galvo-optics machine — contains:

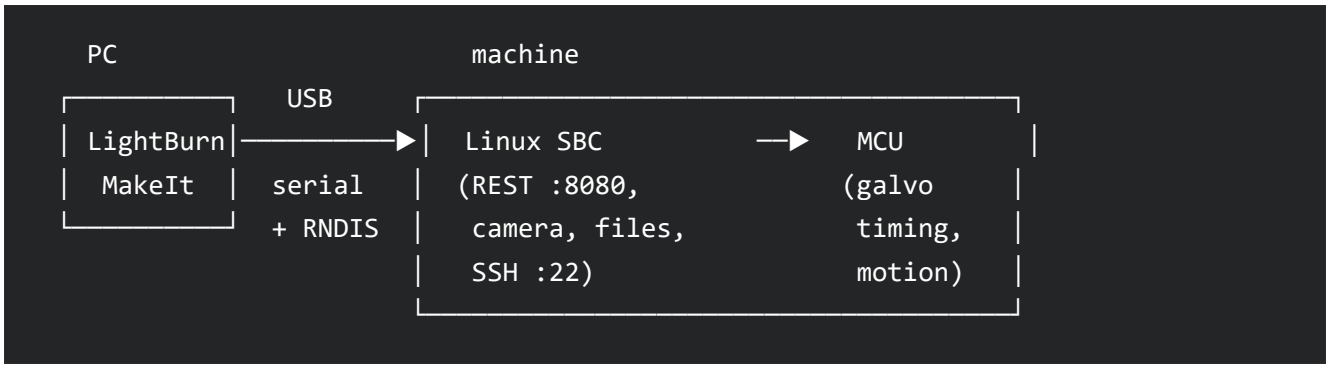
```
"Name": "GRBL",
"Type": "Serial",
"Width": 116,
"Height": 116,
"MirrorY": true,
"Settings": {
    "BaudRate": 1000000,
    "S_Scale": 1000,
    "CutOrigin": 2,
    "EnableZ": false,
    ...
}
```

`"Name"` is the LightBurn **driver type**. `"Type"` is the **connection**. Every one of WeCreat's four published profiles — Vision, Vision Pro, Vista, Lumos — says `"Name": "GRBL"`. **CONFIRMED-vendor.**

LightBurn's own staff have said the same thing about the Lumos in public: it is "some variant of a GRBL device, so the Core version of LightBurn is what you need."

2. Why a galvo machine speaks G-code

Because the galvo timing does not happen on your PC. The controller is a two-tier system:



Evidence for the two tiers:

- WeCreat's bundled RNDIS driver (`resource/driver/win/rndis/rndis11.inf`, shipped inside MakeIt!) binds these hardware IDs: **CONFIRMED-vendor**

```
- USB\VID_0525&PID_a4a2                ; Linux/NetChip USB Ethernet gadget
- USB\VID_1d6b&PID_0104&MI_00          ; Linux Foundation multifunction
  composite gadget
```

  `1d6b` is the Linux Foundation vendor ID and `0104` is the multifunction composite gadget — i.e. **the machine presents itself as a Linux USB gadget**, and `MI_00` means it is a composite device with more than one interface. That is how one cable can carry both a network interface and a serial interface.

- The reverse-engineered REST endpoint for sending a single command is literally **`POST /test/cmd/mcu`** — the SBC forwards to an MCU. **CONFIRMED-community** (via the `weburn` project, Vision generation).
- WeCreat distributes firmware as a **`.tar.gz`** side-loaded through a hidden MakeIt console — a Linux userland payload, not a galvo card image. **CONFIRMED-vendor.**

So LightBurn hands the MCU high-level geometry; the MCU does the galvo interpolation. The machine's headline 16,000 mm/s is therefore *not* evidence against a G-code link for vector work. It is a real concern for **fills**, where LightBurn's GRBL output modulates power per pixel over an ack-per-line link. The Lumos profile's macros `M1S1` ("svg") and `M1S0` ("BMP") look like a controller-side mode switch bolted on for exactly this reason. **INFERRED.**

3. What this costs us

LightBurn gates a large set of features behind its **galvo** device class. On a device typed `GRBL`, they simply do not exist in the UI:

LightBurn feature	Available on a GRBL-typed device?	Consequence for the Ultra
Q-Pulse Width (MOPA ns pulse control)	**No** — galvo-only cut setting	MOPA pulse width is not reachable from LightBurn's UI. Would have to be smuggled in as custom G-code, if an M-code for it even exists
Frequency (kHz) as a cut setting	**No** — galvo-only	Same
Lens correction: **Scale / Bulge / Skew / Trapezoid**	**No** — galvo Device Settings only	Per-lens optical correction cannot be entered. Whatever correction the controller applies internally is what you get
Galvo Rotary Mode (chuck/roller, steps-per-rotation)	**No** — but GRBL rotary **is** available	Use LightBurn's ordinary rotary setup on the A axis. WeCreat's own guide for other models says "Set Axis to **A**"
Split Marking (2.1, galvo-only)	**No**	The Slide Extension cannot be driven by LightBurn's split-marking engine
3D Sliced / depth-map image mode, 16-bit depth maps	**No**	Confirmed failure on the Lumos even with LightBurn Pro. This removes relief/emboss work from LightBurn
Red Dot offsets / rest position	**No**	Framing behaviour is whatever the controller does; `M21S1` may toggle the pointer
Speed, power, passes, interval, hatch angle, cross-hatch	**Yes**	Ordinary GRBL cut settings work normally

LightBurn feature	Available on a GRBL-typed device?	Consequence for the Ultra
Camera	**Yes**, via LightBurn's `LBHTTP:` WeCreat camera path	See 08-cameras.md

For a 100 W MOPA owner this is the sharp edge of the whole project. The reason to want LightBurn for MOPA work is precisely frequency and pulse-width control, and that is the feature LightBurn's GRBL class does not have. Read 09-k9-and-mopa-limits.md before deciding how much to invest here.

4. Confirmed on a physical Lumos Ultra — 2026-08-24

The inference above has been tested and holds. A Lumos Ultra was connected by USB to a Windows 11 machine and enumerated. Both expected devices appeared, and no galvo controller did. **CONFIRMED-hardware.**

```
=== Serial ports ===
  USB-SERIAL CH340 (COM7)
      HWID: USB\VID_1A86&PID_7523\A&336F6178&0&2

=== RNDIS / USB-Ethernet gadgets ===
  USB Ethernet/RNDIS Gadget
      HWID: USB\VID_0525&PID_A4A2\A&336F6178&0&3
```

Full capture: `captures/stage0-usb-benchy-20260824-064202.txt`.

What this establishes:

1. **The Ultra is a GRBL-over-serial device to LightBurn.** `VID_1A86&PID_7523` is a WCH CH340 USB-to-serial bridge. There is **no BJJCZ / JCZ / EZCAD device present** — so LightBurn's galvo device class, and everything gated behind it, is genuinely unavailable. The consequences table in section 3 stands.
2. **The two-tier architecture is real, and the serial link goes to the MCU, not the SBC.** The CH340 is a *discrete* USB-serial bridge, not a CDC-ACM interface on the Linux gadget. So the G-code stream reaches the motion MCU directly, while the SBC sits on its own USB interface. This was previously inference; it is now observed.
3. **Both devices share one internal hub** (instance `A&336F6178`, ports 2 and 3) — one cable into the machine, an internal hub, two independent USB devices behind it.
4. **The RNDIS gadget ID matches WeCreat's own driver INF exactly** (`VID_0525&PID_A4A2`), so the Ultra uses the same Linux USB gadget stack as the Vision and Lumos generations.

5. **The controller has its own private USB network.** The RNDIS adapter came up at **192.168.42.100/24**, which places the controller itself on `192.168.42.0/24` — almost certainly `192.168.42.1`. That is the address to aim Stage 2's HTTP probe at.

Remaining Ultra-specific unknowns are now narrower: the serial banner and baud, whether `$$` is implemented, the Ultra's own M-code numbering, and whether the REST API survived. All are Stage 1 and Stage 2.

5. The two possible integration paths

Path A — native LightBurn GRBL device *(primary)*

Create a GRBL/Serial device profile, connect over the machine's USB serial interface, and drive it the way the Lumos is driven today. This is what `profiles/draft/` targets.

Cost: no camera-driven autofocus beyond an `M130`-style macro, no MOPA pulse control, no relief. Benefit: works today with stock LightBurn, no extra software.

Path B — bridge to the REST API *(secondary, richer)*

The `weburn` project (Vision 20 W only) proved a bridge can present a virtual serial port to LightBurn and translate to the machine's HTTP API on port 8080, gaining camera stills and autofocus probing that the serial path cannot reach. Endpoints documented in 05-connectivity.md.

Cost: extra moving part, null-modem driver, and the REST command endpoint **fires the laser regardless of lid position** — a genuine safety hazard, see SAFETY.md. Benefit: access to camera, autofocus measurement, bulk job upload (which sidesteps the fill throughput problem entirely by letting the controller run the job).

A mature version of this project probably ships **both**: a plain profile for everyday work, and an optional bridge for the features the serial path cannot reach.

6. Open architectural questions

1. Does the Ultra enumerate as a serial device at all, and at what baud? *(Stage 0)*
2. Is the Ultra's serial endpoint the MCU directly, or a CDC-ACM gadget on the SBC? This changes where a bridge would hook in.
3. Are ports 8080 / 8082 / 22 still open on the Ultra, and is the endpoint map unchanged?
4. Does the Ultra implement GRBL `$$` settings at all? No WeCreat machine has ever had its `$$` output published.
5. How is UV↔MOPA source selection commanded? The Lumos uses `M18S1`/`M18S0` for its two sources. The Ultra has UV, MOPA, *and* a red pointer, so the code may differ.
6. Is there any G-code path to MOPA pulse width and frequency?

Each of these has a concrete test in 03-bringup-plan.md.

02 — Feature & Mode Inventory

The master status list. Every Lumos Ultra capability, what it would take to reach it from LightBurn, and where we actually are.

Status vocabulary

Status	Meaning
☐ `NOT-STARTED`	Nobody has looked at it yet
🟨 `DRAFT`	A draft profile or approach exists, untested on hardware
🟧 `NEEDS-HW`	Design is settled; blocked only on someone with the machine running a test
🟥 `BLOCKED-LB`	Blocked by LightBurn — the feature is galvo-class only and this is a GRBL device
🟥 `BLOCKED-FW`	Blocked by WeCreat firmware — the capability is not exposed to third parties
✅ `WORKING`	Confirmed working on a physical Lumos Ultra by at least one contributor

Nothing is ✅ yet. This project is pre-hardware.

A. Laser sources

#	Feature	Detail	Status	Notes
A1	6 W UV source	355 nm, spot 0.0019 mm	🟨 `DRAFT`	Base source, ships with every Ultra
A2	60 W MOPA fiber	JPT module, add-on	🟨 `DRAFT`	Same red lens as A3
A3	100 W MOPA fiber	JPT module, add-on, spot 0.03 mm	🟨 `DRAFT`	

#	Feature	Detail	Status	Notes
A4	UV ↔ MOPA source switching	WeCreat: "one click, no machine swapping"	🔒 `NEEDS-HW`	Lumos uses `M18S1`/`M18S0`. Ultra code **unknown** — do not assume. Open Q3
A5	MOPA frequency (PRR) control		🔒 `BLOCKED-LB`	LightBurn exposes Frequency only on galvo devices. Custom G-code only, if a code exists
A6	MOPA Q-pulse width (ns)	The main reason to want MOPA	🔒 `BLOCKED-LB`	Same. **This is the project's biggest disappointment** — see 09
A7	UV min/max pulse		🔒 `BLOCKED-LB`	Galvo-class `UV Min Pulse` / `UV Max Pulse` fields do not exist for GRBL devices
A8	Power (S value)		🔒 `DRAFT`	Ordinary GRBL. `S_Scale: 1000` per vendor profiles
A9	Speed (F value)		🔒 `DRAFT`	Ordinary GRBL

B. Field lenses

#	Lens	Working area	Status	Notes
B1	**Purple** — general UV, flat & cylindrical	210 × 210 mm	🔒 `DRAFT`	`WeCreat-Lumos-Ultra-UV-Purple-210.lbdev`
B2	**Green** — K9 crystal internal	70 × 70 mm field; 70 × 70 × 135 mm volume	🔒 `DRAFT`	Surface marking through the green lens may work; internal 3D will not — see F4/F5
B3	**Red** — MOPA fiber (60 W and 100 W share it)	210 × 210 mm	🔒 `DRAFT`	Two profiles, one per wattage, so material libraries stay separate

#	Lens	Working area	Status	Notes
B4	Per-lens optical correction (scale/bulge/skew/trapezoid)		□ `BLOCKED-LB`	Galvo Device Settings only. On GRBL the controller's own correction is all you get
B5	Field-lens auto-detection	WeCreat advertises it	□ `BLOCKED-FW`	LightBurn will not switch profiles when you swap a lens. Mitigation: per-profile start-of-job **Checklist** — see 12-checklists.md

C. Work modes

These nine modes are enumerated directly from the shipping MakeIt! 3.0.6 application bundle (`images/lumosUltraWorkMode/`) — this is the vendor's own list for *this machine*, not marketing copy. See 11-makeit-internals.md for method. **CONFIRMED-vendor.**

#	MakeIt mode	What it is	LightBurn equivalent	Status
C1	`plane-mode`	Flat surface marking	Ordinary job	□ `DRAFT`
C2	`cylinder-mode`	Cylindrical objects (rotary)	Rotary setup, A axis	□ `NEEDS-HW`
C3	`emboss-mode`	Relief / depth-map engraving	3D Sliced Image mode	□ `BLOCKED-LB`
C4	`crystal-mode`	K9 internal engraving, 2D layer	—	□ `NEEDS-HW` (probably □)
C5	`crystal-mode3D`	K9 internal volumetric	—	□ `BLOCKED-FW`

#	MakeIt mode	What it is	LightBurn equivalent	Status
C6	`autoslider-mode`	Slide Extension, flat	Split Marking (galvo-only) or manual tiling	☐ `BLOCKED-LB` for automatic; manual tiling ☐ `NEEDS-HW`
C7	`autosliderCylinder-mode`	Slide + rotary, long cylinders	—	☐ `BLOCKED-LB`
C8	`conveyorbelt-mode`	AutoFlow conveyor batch feeding	—	☐ `NEEDS-HW`
C9	`conveyorBeltSlide-mode`	Conveyor + slide combined	—	☐ `BLOCKED-LB`

The generic (non-Ultra) MakeIt mode set also contains `baseplate-mode`, `curve-mode` and `manualhand-mode`; these are not present in the Ultra-specific asset folder and may or may not apply to the Ultra.

D. Accessories

#	Accessory	Spec	Status	Notes
D1	Rotary Pro	Chuck 3–100 mm, sphere/ring 10–37 mm, tilt 0–180°	☐ `NEEDS-HW`	WeCreat lists LightBurn as supported software for the Rotary Pro — but on a page whose compatibility row says "Lumos / Lumos Flex", not Ultra
D2	Slide Extension	520 × 183 mm working area	☐ `NEEDS-HW`	Motor addressing unknown. Automatic split marking is `BLOCKED-LB`
D3	Rotary + Slide combo	Up to 380 mm long, 100 mm dia	☐ `BLOCKED-LB`	Nobody has ever made rotary+slide work in LightBurn on *any* Lumos
D4	AutoFlow Conveyor	10 kg load. Vendor quotes three different working areas — 206 ×	☐ `NEEDS-HW`	Precedent: the Vision passthrough conveyor is

#	Accessory	Spec	Status	Notes
		206 mm/cycle, 200 × 500 mm, and 1000 × 200 mm total feed		driven as an **A axis** through LightBurn's rotary setup
D5	AirGuard Ultra 2.0 extractor		☐ `NOT-STARTED`	Likely independent of LightBurn
D6	Thermoelectric cooling base		☐ `NOT-STARTED`	Likely independent
D7	FreeFlow air assist		▯ `NEEDS-HW`	No WeCreat air-assist M-code has ever surfaced publicly

E. Machine functions

#	Function	Status	Notes
E1	USB connection	▯ `WORKING`	**LightBurn connects and the machine identifies itself.** CH340 at COM7, 1,000,000 baud, `Custom GCode` device with `GCodeFlavor: wecreat` → console reports **"Found WeCreat Device"**. Job streaming still returns `err:20` on two unsupported commands — see capture
E2	Wi-Fi connection	▯ `BLOCKED-FW`	Ultra spec says USB/Wi-Fi, but Wi-Fi is a MakeIt path. No evidence LightBurn can use it
E3	Ethernet	—	Not documented on the Ultra at all
E4	Autofocus	▯ `NEEDS-HW`	Vision family: `M130X<mm>Y<mm>` as a macro or as LightBurn's Focus Z command. Ultra code unverified
E5	Red-dot framing	▯ `NEEDS-HW`	`M21S1` / `M21S0` on the Vision family
E6	Z axis / focus height	▯ `NEEDS-HW`	Vendor Lumos profile has `EnableZ: false`; Vista and Vision Pro have it `true`. Ultra unknown. Drafts ship with Z **disabled** for safety

#	Function	Status	Notes
E7	Homing	☐ `NEEDS-HW`	`HomeOnStartup: false` in every vendor profile
E8	Exhaust fan	☐ `NEEDS-HW`	`M15S1` / `M15S0` on the Vision family — the best-attested WeCreat code after `M130`
E9	Laser module fan	☐ `NEEDS-HW`	`M14S1` / `M14S0` on the Vision family; **absent from the Lumos profile**
E10	Lid interlock	☐ vendor-side	Enforced by the controller, not the software. **The REST command endpoint bypasses it**
E11	"U mode" (`M16` / `M17`)	☐ `UNKNOWN`	Present in the start G-code of *every* WeCreat profile, so it is fundamental. Nobody knows what it is. See 04

F. Cameras & imaging

#	Feature	Status	Notes
F1	Camera positioning in LightBurn	☐ `WORKING`	**Confirmed on hardware 2026-08-24** — the Ultra's camera works in LightBurn 2.1.04. Arrives over RNDIS as an `LBHTTP:` device; one stream, rotated ~90°, lens-uncorrected (see F2)
F2	Number of cameras	☐ **ANSWERED**	**One accessible camera.** `/camera/take_photo` returns a single wide-angle JPEG, not a composite; the controller config names one device, `/dev/video0`; no second endpoint exists. Image arrives **rotated ~90° and lens-uncorrected**. The "16K" figure is an **engraving resolution** claim, not a camera spec
F3	Dual-camera / wall view (LightBurn 2.1)	☐ `BLOCKED-FW`	F2 resolved to one accessible stream — there is no second feed to attach
F4	Smart Fill / batch layout	☐ `BLOCKED-FW`	On-machine feature, no third-party path

#	Feature	Status	Notes
F5	Material recognition	⛔ `BLOCKED-FW`	

G. Cross-cutting LightBurn features

#	Feature	Status	Notes
G1	Per-device start-of-job **Checklist**	🚧 `DRAFT`	Our lens-confirmation safeguard. `"Checklist"` is a top-level key in the `.lbdev`
G2	Material libraries per source	☐ `NOT-STARTED`	Wanted: separate libraries for UV, MOPA 60, MOPA 100
G3	`.lbzip` User Bundle distribution	☐ `NOT-STARTED`	LightBurn 2.x exports `.lbzip`; `.lbdev` is import-only legacy. Bundle can carry devices + material libraries + presets
G4	Fill throughput over serial	⛔ `NEEDS-HW`	Expect fills far below the machine's 16,000 mm/s headline. Needs a benchmark against MakeIt

---

Open questions

1. ~~**Does the Ultra present as a GRBL serial device?**~~ — **ANSWERED 2026-08-24: yes.** A physical Ultra enumerates as a CH340 serial bridge (`VID_1A86&PID_7523`, COM7) plus a Linux RNDIS gadget (`VID_0525&PID_A4A2`), with no galvo controller present. The RNDIS link comes up on `192.168.42.0/24`. See 01-architecture.md §4.
2. ~~**What is the Ultra's serial banner and baud rate?**~~ — **ANSWERED 2026-08-24.** `$I` returns `[WeCreat Lumos :ver 000240]` — note it does **not** say "Ultra" — at **1,000,000 baud**. No unsolicited banner on port open; it is emitted on power-up only. See capture.
3. ~~**What M-code selects UV vs MOPA on the Ultra?**~~ — **ANSWERED 2026-08-24: `M18`.** MakeIt emits `M18S0Q0` with the MOPA source selected — and `M18S0` is exactly what WeCreat's own Lumos profile labels "1064red". The Lumos code carries over to the Ultra. `CONFIRMED-vendor` — see capture.
4. ~~**Is there any G-code for MOPA pulse width / frequency?**~~ — **ANSWERED 2026-08-24: YES.** Two real MOPA jobs differing only in pulse width produced G-code differing by exactly one line: `M39P200` → `M39P500`. **`M39P<n>` is the pulse width; `M38F<n>` is the**

frequency. Deliverable from LightBurn as per-layer custom G-code. `CONFIRMED-vendon` — see capture.

5. **What is "U mode"?** *Test: firmware strings, or differential capture.*
6. ~~**Is the REST API on :8080 unchanged on the Ultra?**~~ — **ANSWERED 2026-08-24: present.** Ports 22, 8080 and 8082 are all open on `192.168.42.1`, `/camera/take_photo` returns an 826 KB JPEG, `/device/camera/download` returns calibration JSON, `/process/status` answers — though its response shape has changed from the Vision generation. **A `weburn`-style bridge is viable on this machine.** See capture.
7. **How are the slide and conveyor motors addressed — A axis, U axis, or remapped Y?** *Test: Stage 7/8 differential capture.*
8. ~~**Does `$$` work?**~~ — **ANSWERED 2026-08-24: no.** The controller acknowledges `$$` with a bare `ok` and returns nothing. `$#` and `$G` are likewise stubs; `?` and `$I` are fully implemented. **Field size and scaling therefore cannot be read from the machine and must be measured** (Stage 4). See capture.
9. ~~**Does the Ultra's field really measure 210 × 210 mm?**~~ — **ANSWERED 2026-08-24: yes.** MakeIt writes `;canvas border: 0 0 210 210` into the header of every job, and pairs it with `M107X-105Y-105` (half of 210) as the origin offset. **The draft profiles are correct.** `CONFIRMED-vendor` — see capture.
10. **Is `Pn:Z` (Z limit asserted at rest) normal, or a stuck input?** Matters before enabling Z or homing.

Each carries a matching issue label. If you can answer one, you have moved this project further than any amount of documentation work.

03 — Staged Bring-Up Plan

Ordered so that the cheapest, safest tests come first and the highest-risk ones come last. Each stage names what you do, what result means what, and where to file it.

Read SAFETY.md first. Stages 0–2 involve no beam at all, and answer most of the project's open questions.

Every stage has a capture template in captures/templates/. File results as an issue or a PR adding a file under captures/.

Stage 0 — USB enumeration *(no beam, no software, 2 minutes)*

This is the single most valuable test in the project. It settles the architecture question.

 # with the Ultra powered on and connected by USB:

powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\probe-workstation.ps1 > .\captures\ultra-usb-<yourname>.txt

On Linux: lsusb, lsusb -v -d <vid:pid>, dmesg | tail -60, ls -l /dev/serial/by-id/, ip -br a.

Interpretation

What you see	What it means
VID_0525&PID_A4A2 or VID_1D6B&PID_0104 under Network adapters	Linux USB gadget — same architecture as the Lumos/Vision. **Expected.**
A COM port alongside it (VID_1A86 CH340, or a CDC-ACM gadget)	The serial path exists. Proceed to Stage 1
A COM port but **no** RNDIS	Simpler than expected; the REST path may be gone
VID_9588 (BJJCZ/JCZ) or a WinUSB device with no COM port	**The Ultra is a real galvo controller and this project's architecture is wrong.** Stop and open an issue immediately — that is a very good outcome, it means LightBurn's galvo features are available
Two COM ports	Known WeCreat quirk — only one is functional

→ File as issue type **Feature status report**, question #1.

⚠ **Before Stage 1: the control-mode latch**

Opening the serial port takes control away from MakeIt until the machine is power-cycled. The controller treats any serial client as "LightBurn" — including a read-only probe script. Nothing is damaged, and it is fully reversible, but you cannot freely alternate between the two programs. If you need MakeIt afterwards (Stage 5 does), power-cycle the machine first. Details and evidence.

Stage 1 — Serial identity *(no beam)*

Talk to the port directly, *before* involving LightBurn, so you see the raw truth.

Windows: PuTTY → Serial, or

python -m serial.tools.miniterm COM<n> 1000000

try also 921600, 500000, 230400, 115200

Power-cycle the machine while connected and capture the banner.

Expect something shaped like [WeCreat Lumos Ultra :ver ######] — the Lumos produces [WeCreat Lumos :ver 000207], plus **extra unsolicited ok lines**. Record the exact string; LightBurn's device detection depends on it.

Then, one at a time, send these **read-only** probes:

Probe	Looking for
?	A real GRBL status frame <Idle|MPos:0.000,0.000,0.000|…>
$$	The settings table. **No WeCreat $$ output has ever been published, for any model.** If it returns nothing, that is itself the finding
$I	Build info
$#	Work coordinate offsets
$G	Parser state

Do not send anything else. Not M1, M6, M16, M17, M18, M19, M57, M107.

A note on M27. Earlier drafts of this document listed M27 among the safe probes, on the grounds that weburn parses a reply shaped M27 X…,Y…,Z… from the Vision. That was inconsistent with our own rule in SAFETY.md — *no blind M-codes* — because WeCreat renumbers M-codes between models and M27 is unverified on the Ultra. It is now **opt-in only**: tools/stage1-serial-probe.ps1 -IncludeM27, which warns and pauses first. The $-commands above are GRBL queries, not WeCreat M-codes, and carry no such risk.

The scripted way

tools/stage1-serial-probe.ps1 does all of the above with nothing but Windows PowerShell — no Python, no PuTTY. It auto-detects the CH340 port, leaves DTR and RTS alone so the controller is not reset, sends only the $ queries and ?, and writes the transcript into captures/.

.\tools\stage1-serial-probe.ps1 *# auto-detect, 1000000 baud*

.\tools\stage1-serial-probe.ps1 -AllBauds *# try each known baud in turn*

→ Template: captures/templates/serial-banner.md

Stage 2 — Network side *(no beam, but see the interlock warning)*

Confirmed on a real Ultra: the RNDIS adapter comes up at **192.168.42.100/24**, which puts the controller on 192.168.42.0/24 — almost certainly **192.168.42.1**. Note this is a *private USB-only* subnet; it is not reachable from anywhere else on your LAN.

tools/stage2-rest-probe.ps1 auto-detects that subnet, scans ports 22/80/8080/8082, and calls only the read-only endpoints. It never calls /test/cmd/mcu, and it skips the head-moving autofocus probe unless you pass -AllowAutofocus.

.\tools\stage2-rest-probe.ps1

.\tools\stage2-rest-probe.ps1 -Target 192.168.42.1

By hand, if you prefer:

curl -X POST "http://<ip>:8080/process/status"

curl "http://<ip>:8080/camera/take_photo" -o test.jpg

curl "http://<ip>:8080/device/camera/download"

curl "http://<ip>:8080/device/camera/measure_distance?x=105&y=105"

 # and check what else is listening

nc -vz <ip> 22 ; nc -vz <ip> 8082

⚠ **Do not use POST /test/cmd/mcu.** That endpoint fires the laser regardless of lid position.

Interpretation: if these answer, the weburn endpoint map survives into the Ultra and a bridge is viable — which is the only route to camera and autofocus features the serial path cannot reach. If 8080 is gone or changed, the real API can be captured with Wireshark on the RNDIS interface while MakeIt runs a job (Stage 5).

→ Template: captures/templates/rest-probe.md

Stage 3 — LightBurn connection, no output *(no beam)*

Import profiles/draft/WeCreat-Lumos-Ultra-UV-Purple-210.lbdev (Laser window → Devices → Import). It ships with empty start/end G-code deliberately.

Connect. Watch the Console.

- Does LightBurn report the machine as connected?

- Does the banner appear?
- Does ? produce sane position?
- Does the Console show "Starting stream" on a job?

Do not run a job yet. If the connection is unstable, note your LightBurn version — 2.1.00 added "Allow for extra ok's from WeCreat firmware" and 2.1.02 reverted the GRBL protocol to "GRBL-ish" specifically for machines like this. Older versions desynchronise and produce the perennial "laser busy" / stuck-at-99 % symptom.

→ Template: captures/templates/lightburn-connect.md

Stage 4 — Motion and framing *(red dot only, no laser)*

With the **purple UV lens installed** and confirmed, and the bed empty:

1. Frame a 10 mm square. Then 50 mm. Then 100 mm. Then 200 mm.
2. Measure the framed rectangle with calipers or a steel rule.
3. Check orientation: does the framed shape match what's on screen, or is it mirrored/rotated?

What you are calibrating: Width/Height, MirrorX/MirrorY, CutOrigin, and ProcessOffsetX/Y in the profile. The vendor Lumos profile uses MirrorY: true and CutOrigin: 2; whether the Ultra matches is unknown.

If the red dot does not come on, try M21S1 — that code is well attested on the Vision family, though not on the Ultra.

→ Template: captures/templates/field-calibration.md

Stage 5 — Differential MakeIt capture *(the M-code goldmine)*

This is how the Ultra's M-code table gets written, and it needs no risky experimentation at all — you simply watch what MakeIt already does.

Capture without firing anything

The controller stores framing G-code separately, under /mnt/SDCARD/framing/. **Framing uses the red pointer, not the working beam.** So a framing capture is completely beam-free and needs no goggles decision and no lens swap — and it still carries the start/end block and the coordinate range, which is where the field size and any M16/M17 will show up.

You do not need to change lenses to start. Use whichever lens is fitted, select the source that matches it, and frame. Never select a source that does not match the installed lens and then let

it fire — the optics are wavelength-specific and the coating is damaged by the wrong wavelength, quite apart from the focus being wrong.

A sensible order:

Capture	Beam?	What it answers
Frame, source matching the fitted lens	**No**	Start/end block, coordinate range → **field size**, whether MakeIt emits M16/M17
A real low-power job on scrap, same lens	Yes	How power, speed, frequency and **MOPA pulse width** are expressed — or whether they are expressed at all
Frame again with the *other* source selected, if MakeIt permits it	**No**	The **source-select code** — tests M104X1 / M104X2

The third row is the one that usually seems to need a lens swap and often does not: switching source in MakeIt and only *framing* never puts a working beam through the wrong optic. If MakeIt refuses to select a source whose lens is not fitted, then and only then is a swap required.

The differential method

Method: run **the same tiny job twice**, changing **exactly one variable**, and diff the G-code. Capture either with Wireshark on the RNDIS interface (grab the body of POST /process/upload) or through MakeIt's hidden console.

Change exactly this	Isolates
UV source → MOPA source	**The source-select M-code** (the Ultra's analogue of the Lumos M18)
MOPA pulse width A → B	**Whether pulse width is expressible in G-code at all**
MOPA frequency A → B	Frequency control
Focus height A → B	Whether focus is a plain G0 Z…
Vector job → raster job	Whether M1S1/M1S0 mode switching survives

Change exactly this	Isolates
Purple lens → green lens	Whether the lens is announced to the controller
Plain job → rotary job	**The rotary axis letter: A, U, or remapped Y**
Plain job → slide job	The slide axis
Plain job → conveyor job	The conveyor feed command
Surface job → K9 internal job	Whether depth layers are Z moves or something else

Post the **diff**, not the whole file, plus the two job settings. →
Template: captures/templates/mcode-discovery.md

Stage 6 — First UV marks *(beam, lowest power)*

Purple lens. Scrap material. UV goggles (190–550 nm). Lowest power / highest speed that marks.

1. A 10 mm square outline.
2. A speed/power grid — LightBurn's Material Test generator.
3. Measure the square. Adjust Width/Height scaling and re-measure.

→ File the resulting numbers as a **calibration data** issue.

Stage 7 — MOPA *(beam, highest risk)*

Red lens. MOPA goggles (800–1100 nm). Metal scrap, never anything reflective and unsecured.

Start absurdly low. Confirm the source actually switched before assuming the settings apply — if UV is still active and you have dialled in MOPA numbers, you will get a surprise.

Establish: does power scale sensibly? Does speed? What are the practical fill rates over the serial link versus MakeIt (Stage 10)?

Stage 8 — Rotary

Precedent from the Vision passthrough conveyor: LightBurn's ordinary **rotary setup on the A axis**. WeCreat's own tumbler guide says "Set Axis to A". The vendor Lumos profile carries rotaryAxis: 3, rotarySteps: 360, rotaryIsChuck: true — rotary settings live **inside the device profile** for GRBL devices, which is convenient for us.

Determine: steps per rotation, direction, whether M16/M17 are involved.

Stage 9 — Slide extension

LightBurn's Split Marking is galvo-only, so automatic slide-driven tiling is not available. What may be possible: driving the slide as a second axis manually, or tiling by hand.

First find out how MakeIt addresses the slide motor (Stage 5).

Stage 10 — Conveyor, and throughput benchmark

Conveyor: feed increment, whether it is bidirectional, and the feed command.

Benchmark: run the same fill in MakeIt and in LightBurn and time both. If LightBurn's serial fill is dramatically slower, the project's centre of gravity should shift toward a bulk-upload bridge rather than live streaming.

Stage 11 — K9 internal engraving

Expected outcome: **not reachable from LightBurn.** LightBurn's depth-map and 3D-slice modes are galvo-class features and are confirmed non-functional on the Lumos even with LightBurn Pro. Internal engraving additionally needs a coordinated focal-depth sweep, not a surface Z.

Worth testing anyway: does ordinary 2D surface marking work through the green lens, at 70 × 70 mm? That would at least make the green lens useful for something in LightBurn.

What to do if you can only do one thing

Stage 0. Two minutes, no beam, and it answers the question everything else depends on.

04 — WeCreat M-code / G-code Dictionary

WeCreat's controllers speak a **private M-code dialect** layered on a GRBL-ish G-code stream. There is no vendor documentation for it. This table is the most complete public compilation we know of, assembled from WeCreat's own shipped `.lbdev` profiles, their support articles, the `weburn` reverse-engineering project, and owner reports.

⚠ The single most important rule

WeCreat renumbers M-codes between machine models. `M14` is in every Vision-family start block and absent from the Lumos. `M18` exists only on the Lumos. `M16`/`M17` mean "Umode" on the Vision but AlgoLaser uses the same numbers for an air pump.

No Lumos M-code may be assumed valid on a Lumos Ultra. Do not paste the Lumos start block into an Ultra profile. This is why `profiles/draft/` ships with **empty** start G-code.

Dictionary

Code	Params	Meaning	Grade	Seen on
`M1`	`S0` / `S1`	Job content-type selector. `S0` labelled **"BMP"** (raster), `S1` labelled **"svg"** (vector). Likely tells the planner whether to expect image-raster or vector geometry. Standard `M1` = optional stop — **repurposed**	Labels CONFIRMED-vendor; meaning INFERRED	Lumos
`M2`	—	End of program. Treated by `weburn` as terminator of a bulk upload	CONFIRMED-community	Vision
`M6`	—	**Job end.** Used as `EndGCode`. Standard `M6` = tool change — **repurposed**	CONFIRMED-vendor	Vision, Vision Pro, Vista
`M14`	`S0` / `S1`	**Laser-module fan** off / on	CONFIRMED-community	Vision, Vision Pro, Vista — **not** Lumos
`M15`	`S0` / `S1`	**Exhaust fan** off / on	CONFIRMED-community	All models
`M16`	—	**"Enable Umode"** — meaning unknown, see below	Label CONFIRMED-	All models

Code	Params	Meaning	Grade	Seen on
			vendor; function UNKNOWN	
`M17`	—	**"DisEnable Umode"** (sic) — inverse of `M16`	Label CONFIRMED-vendor; function UNKNOWN	All models
`M18`	`S0` / `S1`	**Laser source select.** `S0` labelled **"1064red"** (IR), `S1` labelled **"455Blue"** (blue diode)	Labels CONFIRMED-vendor; effect **never publicly verified**	Lumos only
`M19`	`S0`	Present in the Lumos start block, unlabelled	UNKNOWN	Lumos
`M21`	`S0` / `S1`	**Red dot / pointer** off / on	CONFIRMED-community	Vision family
`M27`	—	**Position report.** `weburn` parses a reply shaped `M27 X…,Y…,Z…` and synthesises a GRBL status frame from it	INFERRED (strong — code parses the exact shape)	Vision
`M57`	`A<n>` `B<n>`	Lumos start block, `A130 B120`. **Unknown.** Not the field size — the Lumos field is 116 × 116 mm	UNKNOWN	Lumos

Code	Params	Meaning	Grade	Seen on
`M107`	`X<n> Y<n>`	Lumos start block, `X-60 Y-60`. Best guess: **workspace / field origin offset** — a 116 mm field centred on the optical axis has corners near ±58 mm. Standard `M107` = fan off — **repurposed**	INFERRED	Lumos
`M130`	`X<mm> Y<mm>`	**Autofocus at bed coordinate.** The best-attested WeCreat code — three independent sources. Set as LightBurn's *Focus Z* command or bound to a macro. Standard `M130` = set PID P — **repurposed**	CONFIRMED-community	Vision family
`G0 Z<neg>`		Move to absolute focus height. Z is **negative-down** in machine coordinates (`G0Z-84` ≈ 16 mm object height)	CONFIRMED-community	Vision
`?`	—	GRBL realtime status query	CONFIRMED-community	All
`!`	—	Feed hold / pause	CONFIRMED-community	All
`~`	—	Cycle start / resume	CONFIRMED-community	All
`0x18`	—	Soft reset (Ctrl-X)	CONFIRMED-community	All

Confirmed from MakeIt's own Lumos Ultra G-code, 2026-08-24

Read off the controller after MakeIt generated a framing job — **the vendor's own output**, so these are `CONFIRMED-vendor` rather than inferred. (capture)

```
;wecreat 3.0.6
;canvas border: 0 0 210 210      <- the working area, written into every job
M15S0                            <- exhaust fan off
M107X-105Y-105                   <- field origin offset = half of 210
M41S0
M19S1
M42S0
M18S0Q0                          <- SOURCE SELECT (S) + new Q parameter
M11S0.2
G90
M3S100
#headed
M4S1000
M38F
M39P
G0F480000                        <- 480000 mm/min = 8000 mm/s
```

Code	Observed	Meaning	Grade
`M18`	`M18S0` MOPA · `M18S1` UV	**SOURCE SELECT — confirmed by differential.** Two framing jobs differing only in source produced `S0` vs `S1`, matching WeCreat's own Lumos labels ("1064red" / "455Blue"). On the Ultra, `S1` is the 355 nm UV source	**CONFIRMED -vendor**
`M39`	`M39P200` → `M39P500`	**MOPA PULSE WIDTH.** Two real jobs differing only in MakeIt's pulse-width setting produced G-code differing by exactly this one line. MOPA-only; bare `M39P` in framing jobs	**CONFIRMED -vendor**
`M38`	`M38F48`, `M38F75`	**MOPA frequency.** Varies between jobs, never with pulse width. Bare `M38F` in framing jobs	**CONFIRMED -vendor**
`M5` / `M6` / `M9`	`M5`, `M6`, `M9`	Laser off / job end / coolant off. `M6` matches the Vision family's `EndGCode`. Standard GRBL	CONFIRMED

Code	Observed	Meaning	Grade
`G0 Z`	`G0Z47.8`	**Absolute Z focus move.** Stored default in `/mnt/SDCARD/config/heightZ.txt` = 43.35	**CONFIRMED -vendor**
`M1`	`M1S0`	Content mode — matches the Lumos label "BMP" (raster). `M1S1` = "svg" (vector)	**CONFIRMED -vendor**
`M15`	`M15S0`, `M15S1`, `M15S70`	**Exhaust fan — takes a RANGE, not just on/off.** Earlier evidence had it binary; that was wrong	**CONFIRMED -vendor**
`M42`	`M42S0`, MOPA only	Absent from UV jobs	UNKNOWN
`M41`	`S0` MOPA / `S1` UV	Switches with source — lens or beam path?	UNKNOWN
`M19`	`S1` MOPA / `S0` UV	Also switches with source	UNKNOWN
`M57`	`A450B30` (Ultra) · `A130B120` (Lumos)	Two samples, still unknown	UNKNOWN
`M46`	`A0.9764B20`	`A` near 1.0 reads like a scale factor	UNKNOWN
`M59`	`A-1000B50`		UNKNOWN
`M24` / `M25` / `M26`	`S15`, `S1`, `S1`		UNKNOWN
`M11`	`S0.2` framing / `S0.08` print	Fractional — a physical quantity	UNKNOWN
`M107`	`M107X-105Y-105`	**Field origin offset**, half the field. Lumos: `X-60Y-60` on a 116 mm field. Pattern confirmed across two models — upgraded from INFERRED	**CONFIRMED -vendor**
`M3` / `M4`	`M3S100`, `M4S1000`	**Standard GRBL** laser modes (constant / dynamic power). `M4S1000` confirms `S_Scale:`	CONFIRMED

Code	Observed	Meaning	Grade
		`1000`; per-segment `S` runs 0–900 in a real job	
`G90` / `G4`	`G90`, `G4P1`	Standard absolute positioning, standard dwell	CONFIRMED

`M16`/`M17` **are absent** from MakeIt's framing job — so whatever U-mode is, MakeIt does not need it to frame.

Codes found in Lumos Ultra firmware configuration, 2026-08-24

Recovered from `/etc/mbtc/inverse_distance.json` on the controller's filesystem (capture). **These appear in no WeCreat `.lbdev` and in no prior reverse-engineering work.**

```
"0": { "inverse_dis": "M101X0.001\n",                "inverse_cmd": "M104X1" },
"1": { "inverse_dis": "M44K0.00003273B0.02181818\n", "inverse_cmd": "M104X2" }
```

Code	Observed	Reading	Grade
`M104`	`M104X1`, `M104X2`	Selects between two configurations. The Ultra has exactly two optical paths (UV, MOPA) — a **source/lens select candidate**	**INFERRED**
`M44`	`M44K<n>B<n>`	`K`/`B` read as slope and intercept of a linear fit — a distance calibration	INFERRED
`M101`	`M101X<n>`	Scalar distance parameter, paired with `M104X1`	UNKNOWN

Do not send these to find out what they do. The correct confirmation is to run one job per source in MakeIt and diff the captured G-code — `tools/stage5-jobfile-grab.ps1`.

GRBL `$` command support — measured on a Lumos Ultra, 2026-08-24

CONFIRMED-hardware (capture). The controller implements enough GRBL to stream jobs and report status, and **not** the settings subsystem.

Command	Result	Implemented?
`?`	`<Idle|MPos:0.000,0.000,0.000|FS:0,0|Pn:Z|WCO:0.000,0.000,0.000>`	**Yes** — full GRBL 1.1 status report, three axes
`$I`	`[WeCreat Lumos :ver 000240]`	**Yes**
`$$`	bare `ok`, nothing else	**No**
`$#`	bare `ok`	**No**
`$G`	bare `ok`	**No**

The practical consequence: you cannot read the field size out of the machine. `$130`/`$131` (max travel), `$110`/`$111` (max rates) and `$30` (spindle/S max) do not exist here. Field size and scaling must be established by measurement — see 03-bringup-plan.md Stage 4.

The identity string does not name the model

`$I` returns `[WeCreat Lumos :ver 000240]` on a **Lumos Ultra**. It does not say "Ultra". (The machine's own `/etc/mbtc/devicename.txt` reads `Lumos Ultra`, so the serial string is confirmed to be a **family** string rather than the model.) A Lumos (non-Ultra) reported `[WeCreat Lumos :ver 000207]` in a public forum capture — same product string, same format, older build. Firmware appears to be shared across the family.

⚠ **Trap.** WeCreat's official `WeCreat-Lumos-v1.5.lbdev` will connect to an Ultra without complaint, because nothing in the handshake distinguishes them. That profile declares a **116 × 116 mm** workspace; the Ultra's field is **210 × 210 mm**. Jobs would be silently mis-scaled and mis-placed, with no error.

Baud rate

1,000,000 baud confirmed working on the Ultra, matching the vendor Lumos profile.

Corrections to claims circulating elsewhere

- `M14` **is not a "passthrough/bulk mode" switch.** `M14S0`/`M14S1` are laser-module fan off/on. The passthrough↔bulk switch lives entirely inside the `weburn` bridge program, which watches the serial stream for the literal string `M14S0` that the user manually inserts into LightBurn's start script. It is a property of the bridge, not of the firmware.

- **There is no `M13`.** A placeholder string in `weburn`'s source reads `M13X_Y_`, but the byte array on the same line decodes to `M130X200Y150`. It is a typo.
- **Do not import AlgoLaser's `M16`/`M17` semantics.** AlgoLaser genuinely uses those numbers for an air pump. WeCreat's own label says "Umode", and WeCreat has separate fan codes. The collision is coincidental.

Reconstructed start / end blocks (vendor, verbatim)

```
Vision / Vision Pro / Vista
  StartGCode:  M14S1        ; laser-module fan on
               M15S1        ; exhaust fan on
               M16          ; "Enable Umode"
  EndGCode:    M6           ; job end

Lumos                       (note: no M14, no EndGCode)
  StartGCode:  M15S1        ; exhaust fan on
               M16          ; "Enable Umode"
               M57A130B120  ; UNKNOWN
               M107X-60Y-60 ; field origin offset (inferred)
               M19S0        ; UNKNOWN
  Macros:      M18S1 "455Blue"   M18S0 "1064red"
               M1S1  "svg"       M1S0  "BMP"

Lumos Ultra                 UNKNOWN — this is what the project exists to
determine
```

What is "U mode"?

`M16` / `M17` appear in the start block of **every** WeCreat profile, so they are fundamental — and nobody knows what they do. Hypotheses, with the evidence for and against:

2026-08-24 — new evidence, strongly favouring "USB mode". Opening the Ultra's serial port and sending only read-only GRBL queries caused MakeIt to display: "*Machine is now connected to LightBurn… close the LightBurn software… after restarting the device, reconnect Make It.*" LightBurn was never involved — the controller treats any serial client identically, and **latches control away from MakeIt until the machine is power-cycled.** That observed behaviour is exactly a U/USB-mode handover. See the control-mode latch capture.

Hypothesis	Verdict	Reasoning
A 4th / rotary "U axis"	**Near-refuted**	WeCreat's own rotary guide tells LightBurn users to "Set Axis to **A**". A rotary enable in the unconditional start block of every machine would also be wrong
"USB mode" — hand MCU control from the SBC front-end to the USB serial stream	**INFERRED (strong)** — upgraded from "best fit, unproven"	Explains why it is in every start block, why an inverse exists, and why the architecture needs it (the SBC normally owns the MCU via `/test/cmd/mcu`). **Now matched by observed behaviour**: a serial client takes control and MakeIt loses it until power-cycle. Not yet `CONFIRMED` — we have not seen `M17` restore control, and the latch fired on port open without `M16` being sent, so the firmware may assert it implicitly
"User mode" / unlocked mode	Plausible, unproven	"DisEnable" reads like a literal translation; 用户模式 is a natural candidate
Units mode	**Unlikely**	Units are `G20`/`G21`, and LightBurn has its own modal fields
Air assist	**Unlikely**	That's AlgoLaser. WeCreat labels it "Umode" and has separate fan codes

Decisive test, and it sends nothing: power-cycle, connect MakeIt, run a job, then read the job G-code off the controller (`tools/stage5-jobfile-grab.ps1`) and look for `M16`/`M17` in MakeIt's own output. If the vendor's software emits them around its jobs, the meaning is settled from WeCreat's own G-code rather than by experiment.

How to add a code to this table

1. Get evidence. A vendor label, a support article, a `weburn` constant, or your own differential capture (see 03-bringup-plan.md Stage 5).
2. Open a **M-code discovery** issue with the raw capture attached.
3. Grade it honestly. `INFERRED` is a perfectly respectable entry; a wrong `CONFIRMED` is not.
4. Note which model it was seen on. Model-scoped, always.

We will not accept an entry whose evidence is "I sent it and nothing bad happened." Absence of an observed effect is not a meaning.

05 — Connectivity

What WeCreat documents

Path	Ultra spec sheet	For LightBurn specifically
USB	☑ listed	☑ — WeCreat's LightBurn article: **"Only USB connection is supported."** That article lists Vision 20 W/40 W, Vista 10 W and Vision Pro 45 W, and **predates the Ultra**
Wi-Fi	☑ listed	✕ no evidence. Wi-Fi is a MakeIt path
Ethernet	✕ not documented for the Ultra	—
Bluetooth	✕ not documented as a machine transport	Used by mobile MakeIt for provisioning onto Wi-Fi, not as a control link

LightBurn ↔ Ultra is USB — and the port is now confirmed present on a physical Ultra (see below). Whether LightBurn successfully *talks* over it is Stage 3.

"Serial" here means a COM port over USB — there is no serial cable

Worth stating plainly, because the terminology trips people up:

The Lumos Ultra connects with one USB cable. It has no RS-232 port, and you do not need a serial cable or a USB-to-serial adapter.

When this repository says the machine is a *"serial"* device — and when the .lbdev profiles say "Type": "Serial" — that describes the **software interface**, not the cable. Inside the machine is a **CH340 USB-to-serial bridge chip**. Windows enumerates it over USB and creates a *virtual* COM port for it. On the test machine we used, that was COM7:

USB-SERIAL CH340 (COM7)

HWID: USB\VID_1A86&PID_7523\...

^^^ a USB hardware ID, for a device whose name is "USB-SERIAL"

This is how nearly every hobby laser, 3D printer and GRBL CNC works, which is why LightBurn's connection dropdown offers **"Serial/USB"** as a single combined option. WeCreat's own troubleshooting article tells users to look for USB-Serial CH 340 (COMx) under **Ports** in Device Manager — same thing.

So: one cable, and the COM port it produces is what LightBurn streams G-code over.

What the USB cable actually carries

One cable into the machine, an **internal USB hub**, and **two independent USB devices** behind it — confirmed on hardware, on ports 2 and 3 of the same hub instance:

1. **A CH340 USB-to-serial bridge** — the virtual COM port LightBurn streams G-code over. It wires to the **motion MCU**.
2. **An RNDIS network gadget** — a virtual Ethernet link to the controller's **Linux SBC**. This is how LightBurn's WeCreat camera works (LightBurn 1.7.00: *"Initial WeCreat Vision camera support over RNDIS"*), and it is where the HTTP API lives.

Note these are two *separate devices with different vendor IDs*, not two interfaces of one composite gadget — WeCreat's driver INF describes a composite gadget, but the shipping Ultra uses a discrete CH340 alongside it. The practical upshot is useful: **the G-code path and the SBC path are independent and can be used simultaneously.**

Hardware IDs bound by WeCreat's own bundled driver (MakeIt!/resource/driver/win/rndis/rndis11.inf) — **CONFIRMED-vendor**:

USB\VID_0525&PID_a4a2 ; Linux / NetChip USB Ethernet (RNDIS) gadget

USB\VID_1d6b&PID_0104&MI_00 ; Linux Foundation multifunction composite gadget, interface 0

1d6b:0104 is the generic Linux Foundation multifunction composite gadget. MI_00 confirms a multi-interface device.

Confirmed on a physical Lumos Ultra — 2026-08-24

CONFIRMED-hardware. (capture)

Device	Hardware ID	Role
USB-SERIAL CH340 (**COM7**)	USB\VID_1A86&PID_7523\A&336F6178&0&2	WCH CH340 bridge → the **motion MCU**. This is what LightBurn talks to

Device	Hardware ID	Role
USB Ethernet/RNDIS Gadget	USB\VID_0525&PID_A4A2\A&336F6178&0&3	The **Linux SBC** → camera, REST API

Three things worth drawing out:

1. **The CH340 is a discrete bridge, not a CDC-ACM interface on the Linux gadget.** They are two separate USB devices with different vendor IDs, sitting on ports 2 and 3 of the same internal hub (A&336F6178). So the G-code path and the SBC path are genuinely independent — a bridge program could use both at once.
2. **No galvo controller is present.** No VID_9588 (BJJCZ/JCZ), no driverless device, nothing resembling an EZCAD board anywhere in the enumeration.
3. **The RNDIS link comes up on 192.168.42.0/24**, with the PC at 192.168.42.100. The controller is therefore at (almost certainly) 192.168.42.1. This is a private USB-only subnet — not reachable from the rest of your LAN, and not something to confuse with the machine's Wi-Fi address.

WeCreat's troubleshooting article for the current generation tells users to look in Device Manager for exactly these two: **"USB Ethernet/RNDIS Gadget"** under Network adapters *and* **"USB-Serial CH 340 (COMx)"** under Ports. The Ultra matches that description precisely, so their article applies to it even though it predates the machine.

Driver installation

The RNDIS driver ships inside MakeIt!:

C:\Program Files\WeCreat\makeit-builder\resource\driver\win\rndis\

RNDIS.inf rndis11.inf RNDIS.cat rndis11.cat

usb-driver-installer-x64.exe

usb-driver-installer-x86.exe

Baud rate

WeCreat's published profiles use:

Model	BaudRate
Lumos	**1 000 000**
Vision / Vision Pro / Vista	500 000

The Ultra is untested. Start at 1 000 000 (Lumos lineage), then try 921 600, 500 000, 230 400, 115 200. On a CDC-ACM gadget the baud is nominal and any value works.

Known quirks

- **Two COM ports may appear.** Only one is a real serial link. LightBurn dev comment: *"Should only see 1 com port on Windows. The second isn't truly a com port for serial communication."*
- **"Port failed to open — already in use?"** almost always means **MakeIt is still running** and holding the port. Close it entirely.
- **Extra ok responses.** WeCreat firmware emits unsolicited ok lines around the connect banner. LightBurn 2.1.00 added *"Allow for extra ok's from WeCreat firmware"* and 2.1.02 *"Reverted LightBurn GRBL Protocol to GRBL-ish"* plus a *"WeCreat device detection / messaging fix"*. **Use LightBurn 2.1.02 or newer.** Older builds desynchronise and produce the stuck-at-99 % / "laser busy" symptom.

The REST API (secondary path)

Reverse-engineered on the **Vision 20 W** by the weburn project. Unverified on the Ultra — Stage 2.

Method	Endpoint	Purpose
GET	/camera/take_photo	Single still (4032 × 3024 JPEG on the Vision)
GET	/device/camera/download	Camera calibration JSON — 3×3 point grids at three heights plus focal lengths

Method	Endpoint	Purpose
GET	/device/camera/measure_distance?x=<mm>&y=<mm>	**Autofocus probe.** Returns {"distance": …, "height-z": …}. Errors return 0 for both
POST	/process/status	{"code":0,"status":"","result":0}. 2 or 3 = running
POST	/process/control?action=0	Pause
POST	/process/control?action=2	Cancel and return to 0,0,0
POST	/test/cmd/mcu?md5=<MD5>	⚠ **Execute one G-code command immediately — bypasses the lid interlock**
POST	/process/upload?md5=<MD5>	Bulk-upload an entire job

MD5 scheme: MD5 over the exact request body bytes, sent as an **uppercase** hex digest in the md5 query parameter. A wrong checksum returns a calc_sum field and the command does not run.

Other ports found on the Vision generation: 22 (SSH, root password login permitted) and 8082 (unauthenticated Mongoose directory listing of the whole filesystem, /etc/shadow visible). See SAFETY.md. Treat the machine as hostile on a network.

Why the bridge path is interesting anyway

/process/upload hands the controller a complete job to run on its own. That sidesteps the serial link's per-line acknowledgement entirely — which is where a G-code-streamed **fill** loses most of the machine's 16,000 mm/s headline speed. If Stage 10's benchmark shows a large gap, a bulk-upload bridge is the answer, not a better profile.

Firmware

Distributed as a **.tar.gz**, emailed by support or attached to a support article, and side-loaded through a hidden MakeIt console (Ctrl+Shift+P on the documented Force-OTA path). A published Lumos example is Lumos1.2.2+217+20031.tar.gz (~2 MB).

A tarball is a Linux userland payload — further evidence for the SBC architecture, and the cheapest possible route to a real M-code table: unpack it and run strings | grep -E "^M[0-9]+". **If you can obtain the Ultra's OTA file, that is arguably the highest-value single artifact for this project.** No firmware source, and no GPL offer-of-source page, has been found anywhere on WeCreat's site — which is worth noting given the controller demonstrably runs Linux.

06 — Work Modes, Field Lenses and Working Areas

The nine Ultra work modes

Enumerated from the shipping **MakeIt! 3.0.6** application bundle, which contains a `lumosUltraWorkMode/` asset folder distinct from the generic `workMode/` folder. This is the vendor's own mode list for *this specific machine*. **CONFIRMED-vendor** — method in 11-makeit-internals.md.

MakeIt asset	Mode	Requires
`plane-mode`	Flat surface engraving	—
`cylinder-mode`	Cylindrical / rotary engraving	Rotary Pro
`emboss-mode`	Relief / depth-map engraving	—
`crystal-mode`	K9 crystal internal engraving	Green lens
`crystal-mode3D`	K9 volumetric 3D internal engraving	Green lens
`autoslider-mode`	Slide Extension, flat work	Slide Extension
`autosliderCylinder-mode`	Slide + rotary, long cylinders	Slide + Rotary
`conveyorbelt-mode`	AutoFlow conveyor batch feeding	Conveyor
`conveyorBeltSlide-mode`	Conveyor + slide combined	Conveyor + Slide

The generic (non-Ultra) MakeIt mode set additionally contains `baseplate-mode`, `curve-mode` and `manualhand-mode`. These have no Ultra-specific asset and may or may not apply.

Two observations worth drawing out:

1. **`crystal-mode` and `crystal-mode3D` are separate modes.** That supports the hypothesis that flat/2D internal marking is a distinct, simpler operation from volumetric 3D — and therefore that 2D crystal work is the more plausible thing to reach from LightBurn. Worth testing (Stage 11) even though 3D almost certainly is not.
2. **The slide and conveyor each have a "plain" and a "combined" mode.** So the controller distinguishes slide-alone from slide+rotary and conveyor-alone from conveyor+slide. Four distinct motion configurations to characterise, not two.

Field lenses

Lens	Purpose	Working area	Notes
Purple	General UV — flat *and* cylindrical	210 × 210 mm	WeCreat's own wording is "flat or cylindrical", not merely "general". Reviewer-reported focal length 200 mm
Green	K9 crystal internal engraving	70 × 70 mm	Volume 70 × 70 × 135 mm. Reviewer-reported 70 mm focal length
Red	MOPA fiber (60 W and 100 W share it)	210 × 210 mm	Reviewer-reported 200 mm focal length

Vendor "Working area" spec row, verbatim:

Standard Work Area: 210*210mm / Crystal Inner Engraving: 70*70mm / Slide Extension: 520mm*183mm / Auto Conveyor: 1000mm*200mm (Batch Feeding)

Max processing height row: **Surface Engraving 130 mm / Inner Engraving 160 mm.** Note the mild inconsistency with the Kickstarter FAQ's 135 mm crystal engraving depth — 135 mm is plausibly the usable engraving depth and 160 mm the mechanical clearance.

Lens auto-detection

WeCreat advertises **Field-Lens Auto Detection**, and MakeIt uses it. LightBurn cannot: it will not switch device profiles because you physically swapped a lens.

This is a safety issue, not just an inconvenience. Marking with the wrong profile means the wrong field size, and — if the source also differs — potentially the wrong goggles.

Our mitigation is LightBurn's per-device **Checklist** (a top-level `"Checklist"` string in the `.lbdev`), which pops before every job. See 12-checklists.md.

Why separate profiles rather than one

LightBurn's own guidance for interchangeable optics is one device profile per lens, because field dimensions and calibration differ. For this machine the argument is even simpler: the working area is different (210 vs 70), and on a GRBL device the workspace size *is* the profile.

Splitting 60 W and 100 W MOPA into separate profiles despite identical geometry buys one thing: **separate material libraries and separate checklists**. A 100 W setting run on a 60 W machine underperforms; a 60 W setting run on a 100 W machine over-drives. Keeping them apart is worth a duplicate profile.

Profile matrix

Profile	Source	Lens	Width × Height
`WeCreat-Lumos-Ultra-UV-Purple-210`	6 W UV	Purple	210 × 210
`WeCreat-Lumos-Ultra-UV-Green-K9-70`	6 W UV	Green	70 × 70
`WeCreat-Lumos-Ultra-MOPA60-Red-210`	60 W MOPA	Red	210 × 210
`WeCreat-Lumos-Ultra-MOPA100-Red-210`	100 W MOPA	Red	210 × 210
`WeCreat-Lumos-Ultra-Slide-520x183`	any	any	520 × 183
`WeCreat-Lumos-Ultra-Conveyor-206`	any	any	206 × 206

The last two are **speculative** — they assume the machine can be told to treat a larger area as its workspace, which is unproven. They exist so that a contributor with the hardware has something to test rather than something to build.

Reported working areas for the conveyor disagree across WeCreat's own pages: 206 × 206 mm per cycle (Kickstarter FAQ), 200 × 500 mm (conveyor product page), 1000 × 200 mm (main spec table). Most plausible reading — 206 × 206 mm is the per-pass galvo field on the belt, 500 mm the initial load area, 1000 mm the total fed length — but that is **INFERRED**, not confirmed.

07 — Accessories: Rotary, Slide, Conveyor

Rotary Pro — the most promising accessory

Specs (vendor): chuck 3–100 mm diameter, sphere 10–37 mm, ring 10–37 mm, tilt 0–180°, 247 × 81 × 101 mm, 0.971 kg. With the Slide Extension: up to 380 mm long.

Vendor lists LightBurn as supported software for the Rotary Pro — the spec row reads literally Software WeCreat MakeIt! LightBurn. Caveat: that page's compatibility row says "Lumos / Lumos Flex", not Ultra, even though a "Rotary Pro for Lumos Ultra" variant is sold.

Why this is the most likely accessory to work: on a GRBL-typed device, LightBurn's ordinary rotary support applies, and **rotary settings live inside the device profile**. WeCreat's own Lumos profile carries them:

"rotaryAxis": 3,

"rotarySteps": 360,

"rotaryDiameter": 86,

"rotaryIsChuck": true,

"rotaryMode": false,

"mirrorRotaryOutput": false

WeCreat's own tumbler guide for other models instructs LightBurn users to **"Set Axis to A"**. The Vision passthrough conveyor is likewise driven through LightBurn's rotary setup as an A axis with 360 mm per rotation.

What needs determining on the Ultra: the axis letter, steps per rotation, direction, and whether any enable command is required.

Marketing caution: WeCreat also advertises "34-inch baseball bats". 34 in = 864 mm, which contradicts the 380 mm figure given for rotary+slide. Treat the bat claim as belonging to a different machine.

Slide Extension

Specs (vendor): 520 × 183 mm working area, "2.2× expanded workspace", precision linear screw drive. $349 pre-order, shipping from September 2026.

The problem. LightBurn's mechanism for engraving beyond a galvo field is **Split Marking**, added in 2.1 — and it is a **galvo-class feature**. On a GRBL-typed device it does not exist. Split Marking also requires the linear table to be driven by the controller's own motor output with steps-per-unit configured in LightBurn, which is a galvo device-settings screen.

What may still be possible:

- Driving the slide as an additional axis through custom G-code, once its axis letter is known.
- Manual tiling: run the left half, advance the slide by a known amount with a macro, run the right half. Crude, but it works and needs nothing from LightBurn.
- A bridge (Path B) that intercepts and orchestrates.

Prior art is discouraging. Nobody has publicly made rotary + slide work in LightBurn on any Lumos. LightBurn staff, July 2025: they asked WeCreat for device configuration and firmware documentation and *"didn't hear back."* WeCreat's native software reportedly restricts rotary+slide to unidirectional movement.

First step: Stage 5 differential capture — run a slide job in MakeIt and find the axis word.

AutoFlow Conveyor

Specs (vendor): 10 kg load capacity, plug-and-play, "SmartFill for Batch Automation" using the 50 MP camera. $499 pre-order, shipping from September 2026.

Working area — vendor sources disagree:

Source	Figure
Main product spec table	1000 × 200 mm (Batch Feeding)
Conveyor product page	up to 200 × 500 mm
Kickstarter FAQ	206 × 206 mm per cycle

Most plausible reconciliation: 206 × 206 mm is the per-pass galvo field on the belt, 500 mm the initial loading area, 1000 mm the total fed length. **INFERRED.**

Precedent: the Vision's passthrough conveyor *is* usable from LightBurn — WeCreat published a guide for it, driving the feeder through LightBurn's rotary setup on the **A axis** with preset parameters. If the Ultra's conveyor is addressed the same way, this is more reachable than the slide.

What is almost certainly not reachable: SmartFill camera-driven batch layout, item detection, and camera-triggered registration. Those are on-machine features with no third-party interface.

Unknowns: feed increment per cycle, whether the belt is bidirectional or advance-only, and the feed command itself.

Safety note. The conveyor exists to run long unattended batches. See SAFETY.md.

The M16 / M17 "U mode" question matters here

Those two codes appear in the start block of every WeCreat profile. One live hypothesis is that "U" is a fourth axis. WeCreat's own rotary documentation says the rotary axis is **A**, which mostly refutes that — but it leaves the slide and conveyor unaccounted for. If either turns out to be a "U" axis, M16/M17 acquire an obvious meaning.

A Stage 5 capture of a slide job and a conveyor job would settle both questions at once.

08 — Cameras

Good news first

LightBurn already has WeCreat camera integration. This is one of the few areas where the groundwork is done for us:

- LightBurn **1.7.00**: *"Initial WeCreat Vision camera support over RNDIS."*
- LightBurn **2.1** ships a built-in **WeCreat camera preset** (alongside Creality Falcon A1 Pro, Thunder Laser Vision, xTool P2).
- LightBurn maintains a dedicated guide: *WeCreat Camera Calibration and Alignment*.
- The Lumos's internal camera is confirmed working in LightBurn today by owners (rotated 90°, fixed via Camera Alignment).

WeCreat's own FAQ: *"Why am I unable to see or select WeCreat camera?"* → *"Please ensure the LightBurn version is 1.7.00 or above."* Camera support additionally needs machine firmware ≥ 2.0.24.

How it works

The camera is **not a USB webcam**. It arrives over the RNDIS virtual network link as an HTTP image source. LightBurn's device profiles name it with an LBHTTP: prefix — the vendor Lumos profile carries "LastCamera": "LBHTTP: WeCreat Vision", and the Vision Pro profile "LBHTTP: WeCreat Camera".

LBHTTP: is LightBurn's internal "fetch JPEG stills over HTTP" camera driver. It almost certainly maps to GET http://<rndis-ip>:8080/camera/take_photo — the only endpoint that returns an image, and the resolutions line up. **INFERRED**, not documented by LightBurn.

Critical setup detail: LightBurn's lens calibration must be set to **"(None – precalibrated camera)"**, because the WeCreat controller corrects lens distortion itself before serving the frame. Running LightBurn's own lens correction on top double-corrects and produces nonsense. The controller's calibration data is retrievable at GET /device/camera/download (3×3 point grids at three heights, plus focal lengths).

Frame rate is low — the weburn author measured roughly 3–4 fps polling stills.

How many cameras does the Ultra have? — ANSWERED

One accessible camera. CONFIRMED-hardware, 2026-08-24.

Three independent observations on a physical Ultra (capture):

1. GET http://192.168.42.1:8080/camera/take_photo returns **a single 826 KB JPEG** — one wide-angle frame of the bed, **not** a composite or a stereo pair.
2. The controller's own /etc/mbtc/cfgs.json names exactly one capture device: "videoid":"/dev/video0".
3. No second camera endpoint was found on the API.

If the Ultra contains more than one physical sensor, **only one is exposed**, and only one is therefore usable from LightBurn or from a bridge. LightBurn 2.1's dual-camera "wall view" has nothing to attach a second feed to.

Two further observations from the captured frame, both matching known Lumos behaviour:

- **The image is rotated roughly 90°** relative to the bed. Lumos owners report the same, and fix it with LightBurn's Camera Alignment rather than by rotating the source.
- **The frame is raw wide-angle and visibly barrel-distorted** — the enclosure walls curve. So this endpoint serves an *uncorrected* image, even though LightBurn's WeCreat workflow expects a precalibrated feed and tells you to set lens calibration to "(None – precalibrated camera)". Whether LightBurn's LBHTTP: path requests a different, corrected stream is **UNKNOWN** and worth establishing before trusting camera alignment.

What WeCreat publishes, for contrast

Every published reference is singular: "HD Camera" in the spec table, "Built-in 50 MP Camera" in listings.

Two figures get conflated in coverage of this machine, and it is worth separating them:

Figure	What it actually refers to
50 MP	The camera, specifically in connection with **Smart Fill** batch layout
16K	**Engraving/image resolution** — it appears under the spec row "Image Resolution" and in the heading "Ultra-fine 16K Precision Engraving". **It is not a camera spec**

If the Ultra does have two internal cameras, the question that matters for LightBurn is whether they present as **two separately addressable streams** or are merged by the controller into one composite image. LightBurn 2.1 added multi-camera and "wall view" support, so two streams would be usable in principle — but only if the controller exposes them.

This is testable at Stage 2: poll /camera/take_photo and see what comes back, and look for any second camera endpoint.

Camera capabilities worth having, and their odds

Capability	Odds from LightBurn	Why
Camera overlay / positioning	**Good**	Preset exists, precedent on the Lumos
Camera-assisted alignment of artwork to a part	**Good**	Standard LightBurn camera workflow
Autofocus measurement at a point	**Fair**	/device/camera/measure_distance exists; reachable via the bridge, or via an M130-style macro on the serial path
Dual-camera wall view	**Unknown**	Depends entirely on whether two streams exist

Capability	Odds from LightBurn	Why
Smart Fill / batch auto-layout	**None**	On-machine feature, no third-party interface
Material recognition	**None**	Same

What to capture

If you have an Ultra:

1. Does the WeCreat preset in LightBurn 2.1 find the Ultra's camera at all?
2. What does GET http://<ip>:8080/camera/take_photo return — resolution, orientation, one image or a composite?
3. Does GET /device/camera/download return calibration, and what does its structure look like?
4. Is there any endpoint that yields a *second* camera?

→ Template: captures/templates/rest-probe.md

09 — The Two Hard Walls: MOPA Pulse Control and K9 Internal Engraving

Two capabilities that make the Lumos Ultra interesting are the two least likely to be reachable from LightBurn. Both fail for the *same underlying reason*, and it is not a missing config file.

Wall 1 — MOPA frequency and Q-pulse width

Why you want it

MOPA's whole point over a Q-switched fiber laser is independently controllable **pulse width** and **repetition rate**. That is what gives you colour marking on stainless, controlled heat input on brass, and clean deep relief without slag. Without it a 100 W MOPA behaves like an expensive ordinary fiber laser.

Why LightBurn can't give it to you here

LightBurn *does* support these parameters — thoroughly. In the Cut Settings Editor it exposes **Frequency** (kHz) and **Q-Pulse Width** (ns), and in galvo Device Settings **Enable Q-Pulse**

Width Setting, **Fiber Type**, **MO Enabled ONLY When Running**, **Open MO Delay**, **UV Min Pulse**, **UV Max Pulse**.

Every one of those is a galvo-device-class field. On a device typed "Name": "GRBL", LightBurn's cut settings are speed, power, passes, interval, and the ordinary GCode set. There is no frequency field and no pulse-width field, because GRBL has no concept of them.

So the limitation is structural: it is not that WeCreat forgot to ship a profile, it is that the device class LightBurn talks to this machine with does not carry these parameters.

✅ **RESOLVED 2026-08-24 — this wall is DOWN**

MOPA pulse width and frequency ARE controllable from LightBurn, via per-layer custom G-code.

Two real MOPA jobs, identical except for the pulse-width setting in MakeIt, produced captures differing by **exactly one line**:

- M39P200

+ M39P500

M38F<kHz> ; MOPA frequency CONFIRMED-vendor

M39P<ns> ; MOPA pulse width CONFIRMED-vendor

M18S0 ; MOPA source select CONFIRMED-vendor

LightBurn's GRBL device class supports **per-layer custom G-code** — exactly the granularity a MOPA material library needs. Each layer can carry its own M38/M39 pair.

So the limitation is narrower than it appeared: **LightBurn's galvo-class *UI fields* remain unavailable, but the underlying capability is not.** You lose the convenient spin-boxes; you keep the control.

Caveat before building on this: the mapping between MakeIt's UI units and the G-code value has not been verified, and the accepted ranges are unknown. See the capture.

The rest of this section documents *why* the native fields are unavailable, which is still accurate and still worth understanding.

What remains to establish

1. **Unit mapping.** P200/P500 and F48/F75 are MakeIt's own numbers. Nanoseconds and kilohertz are the natural reading and fit typical JPT MOPA ranges, but the relationship between MakeIt's UI field and the emitted value is unverified.

2. **Accepted ranges**, so a profile does not offer values the firmware rejects.
3. **Emitting them from LightBurn**, which is untested. The codes are confirmed as *WeCreat's output*; sending them *to* the machine ourselves is a separate step.

Until those three are settled the profiles ship without them — this project's rule is that a profile contains nothing that has not been demonstrated end to end.

Wall 2 — K9 crystal internal engraving

What it needs

Internal 3D engraving is not surface marking with a different field size. It requires coordinated X/Y galvo position, **Z focal depth**, pulse energy, point timing, and layer sequencing to place discrete fracture points inside a volume. WeCreat describes proprietary volumetric depth mapping and adaptive focal control.

Why LightBurn can't do it here

Two independent blocks:

1. **LightBurn's 3D / depth-map engraving is galvo-class.** *3D Sliced Image Mode*, 16-bit depth maps, 256+ pass cleanup — all galvo-only. This is confirmed by failure on the actual Lumos: an owner with **LightBurn Pro** reported depth-map images simply do not work, and the community explanation is exact — *"it uses GRBL, so it works with the core version... it's not a real galvo control laser and can't use depth map due to that."*
2. **Even with depth maps, internal engraving is a different operation.** A depth map drives a *surface* Z. Internal engraving drives a *focal plane sweep inside a transparent solid*. LightBurn has no model for the latter.

MakeIt exposes crystal-mode and crystal-mode3D as two distinct modes, and accepts STL / PLY / OBJ / GLB input for them — a 3D-mesh pipeline LightBurn does not have.

The consolation prizes

- **2D internal marking** (crystal-mode, not crystal-mode3D) is a simpler operation and might be reachable. Untested.
- **Ordinary surface marking through the green lens** at 70 × 70 mm should work fine, giving you a fine-detail small-field UV profile. That alone justifies shipping the green profile.

Both are Stage 11.

Wall 3 (bonus) — relief / emboss

emboss-mode is in MakeIt's Ultra mode list, and it fails for exactly the same reason as K9 3D: LightBurn's depth-map engraving is galvo-class and confirmed non-functional on this controller family.

For anyone doing 16-bit depth-map relief work — brass coin blanks and the like — **that work stays in MakeIt for now**, or moves to a bridge that uploads a controller-native job.

How to falsify all of this

Every claim above rests on one inference: *the Ultra behaves like the Lumos*. If Stage 0 shows the Ultra enumerating as a **JCZ/BJJCZ galvo board**, then LightBurn's entire galvo feature set — Q-pulse width, frequency, lens correction, split marking, 3D depth maps — becomes available and this document is wrong in the best possible way.

That is a two-minute test. Please run it.

10 — The `.lbdev` File Format

Two corrections to widely repeated assumptions, up front:

1. **`.lbdev` is JSON, not XML.** There is no root element.
2. **LightBurn 2.x no longer *exports* `.lbdev`.** The Devices window exports a `.lbzip` User Bundle. `.lbdev` remains **import-only** — which is exactly why vendors still ship it.

Structure

A `.lbdev` is a UTF-8 JSON object, pretty-printed with 4-space indent, whose only top-level key is `DeviceList` — an **array**, so one file can carry several profiles. LightBurn's import dialog confirms this: *"Select a `.lbzip`, `.lbdev`, or `.lbvendor` file containing one or more device profiles."*

```
{
    "DeviceList": [
        {
            "Checklist": "",
            "DefaultCutList": [ ... ],
            "DefaultToolCutList": [],
```

```
            "DisplayName": "WeCreat Lumos Ultra - UV Purple 210",
            "GUID": "AbCdEfGhIj",
            "Name": "GRBL",
            "Type": "Serial",
            "Width": 210,
            "Height": 210,
            "MirrorX": false,
            "MirrorY": true,
            "ProcessOffsetX": 0,
            "ProcessOffsetY": 0,
            "EnableProcessOffset": false,
            "HomeOnStartup": false,
            "LastCamera": "",
            "Settings": { ... }
        }
    ]
}
```

Top-level keys that matter

Key	Meaning
`Name`	**The LightBurn driver type.** `"GRBL"` for every WeCreat machine. Galvo devices would say `"JCZFiber"`, `"BSLFiber"` or `"EZCad3"`
`Type`	**The connection type.** `"Serial"`, `"TCP"`
`DisplayName`	What the user sees in the Laser window
`GUID`	A short identifier string (10 chars in observed files). Must be unique per profile
`Width` / `Height`	Workspace in mm. On a GRBL device this *is* the field size
`MirrorX` / `MirrorY`	Output mirroring. All WeCreat profiles use `MirrorY: true`

Key	Meaning
`ProcessOffsetX` / `Y` + `EnableProcessOffset`	Rigid offset of the job within the field
`Checklist`	**Free text shown before every job.** Our lens/goggle safeguard
`LastCamera`	e.g. `"LBHTTP: WeCreat Vision"`
`DefaultCutList`	Preloaded layer settings. May be an empty array
`Settings`	Flat sub-object, ~90 driver-dependent keys

`Settings` keys observed in WeCreat profiles

Key	Lumos value	Note
`BaudRate`	`1000000`	Vision family uses `500000`
`CommPort`	`"COM3"`	**Vendor leftovers** — should be `"(Choose)"` in a clean profile
`S_Scale`	`1000`	Max S value
`CutOrigin`	`2`	Job origin corner
`EnableZ`	`false`	`true` on Vista and Vision Pro
`RelativeZOnly`	`false`	`true` on Vision
`StartGCode` / `EndGCode`	see 04	
`Macro0..5_Label` / `_Content`	six macro buttons	
`rotaryAxis`	`3`	**Rotary config lives in the device profile** on GRBL devices
`rotarySteps`	`360`	
`rotaryDiameter`	`86`	

Key	Lumos value	Note
`rotaryIsChuck`	`true`	
`rotaryMode`	`false`	Rotary off by default
`Sim_MaxSpeedX` / `Y`	`500` / `400`	Preview simulation only
`Sim_MaxAccelX` / `Y`	`3000`	
`AirAssistM7`	`false`	
`EnableGrblJCommand`	`false`	
`ForceSValueOutput`	`false`	
`GCodeClustering`	`false`	Worth testing for fill throughput
`LastMachineFilePath`	`"C:/Users/ckr-pc/Desktop"`	**Vendor leftover** — WeCreat shipped their own build machine's path
`DockState_*`		UI layout, harmless

Hygiene rules for profiles in this repo

The vendor files are visibly dumped from a working install and never cleaned. Ours must be clean, because a profile is a public artifact:

- `CommPort` → `"(Choose)"`, never a hard-coded COM number
- `LastMachineFilePath`, `LastDevLibraryPath` → `""`
- `DisplayName` → the real name, not `"WeCreat Lumos (3) (1) (1)"`
- `Info` → `""`
- Unique `GUID` per profile
- `StartGCode` / `EndGCode` / macros → **empty**, until verified on hardware
- `EnableZ` → `false` until verified

Related formats

Extension	What it is
`.lbdev`	Device profile(s). **Import-only** in 2.x

Extension	What it is
`.lbzip`	**User Bundle** — settings, hotkeys, material test presets, image presets, devices, material libraries, art libraries. `File → Bundles → Export/Import Bundle`, or drag the file into LightBurn. *Not* project files
`.lbvendor`	Vendor Bundle — like a User Bundle plus banner image, device icon, vendor name/about/website/support fields, and (2.1) Vendor Menus
`.lbset`	**Controller firmware settings** dump from Machine Settings. Not a device profile
`.lbprefs`	Exported application preferences
`prefs.ini`	LightBurn's live preferences. Despite the extension **it is JSON**, and its `DeviceList` array has exactly the same shape as a `.lbdev`

That last point is genuinely useful, and it is the practical way to learn what LightBurn actually writes: **a `.lbdev` is effectively the `DeviceList` slice of `prefs.ini` written to its own file.** So to discover the real key names for any device configuration — including ones the documentation does not describe — build the device in LightBurn's wizard and then read your own `prefs.ini`, rather than trying to export it.

You cannot export a `.lbdev` from LightBurn 2.x. The Devices window's Export button produces a `.lbzip` User Bundle. `.lbdev` is import-only. If you are trying to capture a device configuration, either export the `.lbzip` (it is a zip — the device JSON is inside) or just copy `prefs.ini`. `tools/test-machine.ps1` menu option `p` does the latter automatically.

`prefs.ini` is your entire LightBurn configuration — every device, path and preference. This repository gitignores it and publishes only extracted key names.

On Windows, LightBurn's preferences folder for recent builds is `%LOCALAPPDATA%\LightBurn\` (containing `prefs.ini`, `backup/`, `themes/`). Use **File → Preferences → Open Prefs Folder** to be certain.

Distribution plan for this project

Ship `.lbdev` files in `profiles/draft/` for review and diffing — they are plain JSON and behave well in git. Once profiles are confirmed on hardware, also publish a single `WeCreat-Lumos-Ultra.lbzip` User Bundle carrying all profiles plus per-source material libraries, so users import once.

11 — MakeIt! Internals

What can and cannot be learned by inspecting a locally installed copy of WeCreat MakeIt!.

Purpose and scope. Everything here is done for **interoperability** — determining the interface the operator's own hardware expects, so a LightBurn profile can be written for it. This repository publishes only the resulting **factual findings**. No WeCreat code, asset, or extracted file is redistributed. `reference/` is gitignored.

Findings below are from **MakeIt! 3.0.6** on Windows.

Layout

```
C:\Program Files\WeCreat\makeit-builder\
    WeCreat MakeIt!.exe
    resources\
        app.asar                ~417 MB   Electron application archive
        app-update.yml
    resource\
        driver\win\rndis\       RNDIS.inf, rndis11.inf, .cat files,
                                usb-driver-installer-x64.exe / -x86.exe
    locales\  *.pak             Chromium locale bundles
```

It is an **Electron** application (Chromium `.pak` files, `app.asar`, `LICENSE.electron.txt`).

Inside `app.asar`:

```
packages/laser-builder/main/dist/index.jsc         873 KB   V8 bytecode
(bytenode)
packages/laser-builder/main/dist/f.enc            ~41 MB   ENCRYPTED
packages/laser-builder/main/dist/opencv.js         ~8 MB
packages/laser-builder/preload/dist/index.jsc      336 KB   V8 bytecode
packages/laser-builder/renderer/dist/...                    assets, images,
fonts, models
node_modules/...                                            ordinary dependencies
```

Finding 1 — The application logic is protected

The renderer's JavaScript is **not present as `.js` in the archive**. The main and preload bundles are compiled to **V8 bytecode** (`.jsc`, bytenode), and the bulk of the application is in **`f.enc`, which is encrypted**.

We confirmed this by scanning the extracted `.jsc` files for machine vocabulary, G-code tokens and HTTP endpoints. Results were empty — the only strings recovered were generic (`http://localhost`, `http://127.0.0.1`, JSON-schema URLs, font names).

Consequence: static analysis will not yield the Ultra's M-code table. The machine definitions are behind encryption. The practical route to the Ultra's protocol is therefore **live capture** — Wireshark on the RNDIS interface while MakeIt runs a job, or the firmware tarball. See 03-bringup-plan.md Stage 5.

This is worth stating plainly so nobody else burns a weekend on it.

Finding 2 — The Ultra's work-mode list, from the vendor's own assets

The renderer's image tree is *not* encrypted, and it is organised per machine. MakeIt 3.0.6 contains a `lumosUltraWorkMode/` folder distinct from the generic `workMode/` folder:

```
renderer/dist/images/lumosUltraWorkMode/
    plane-mode.png
    cylinder-mode.png
    emboss-mode.png
    crystal-mode.png
    crystal-mode3D.png
    autoslider-mode.jpg
    autosliderCylinder-mode.jpg
    conveyorbelt-mode.png
    conveyorBeltSlide-mode.png

renderer/dist/images/lumosUltraTip/
    tip1.png  tip2.png  tip3.png  tip4.png
```

Compare the generic set, which additionally has `baseplate-mode`, `conveyor-mode`, `curve-mode` and `manualhand-mode`.

This is the authoritative Lumos Ultra mode inventory — the vendor's own list for this machine, not marketing copy. It is the basis of section C of 02-feature-inventory.md.

Finding 3 — The updater points at a GitHub repository

`resources/app-update.yml`:

```
owner: yuerugoutql
repo: makeit-builder
provider: github
updaterCacheDirName: makeit-builder-updater
```

MakeIt updates itself from a GitHub repository, `yuerugoutql/makeit-builder`. If that repository is public, its **releases** would carry every MakeIt build — including older ones — and release notes may name machines and features before the support-site changelog does. Worth checking; we have not been able to confirm its visibility.

Finding 4 — Version drift

The installed build is **3.0.6**. WeCreat's public "Software Version Information" changelog tops out at **3.0.2**, and never mentions the Lumos Ultra at all. The shipping software is ahead of the published changelog — so the changelog is not a reliable guide to what MakeIt supports.

Reproducing this

`tools/asar-tool.js` is a dependency-free reader for Electron `.asar` archives:

```
$node = 'C:\Program Files\nodejs\node.exe'
$asar = 'C:\Program Files\WeCreat\makeit-builder\resources\app.asar'

# what's inside
& $node .\tools\asar-tool.js list $asar
& $node .\tools\asar-tool.js list $asar "lumosUltra"

# pull one file out (into gitignored reference/)
& $node .\tools\asar-tool.js extract $asar "packages/laser-
builder/main/dist/index.jsc" .\reference\makeit-extract\main-index.jsc

# search text-ish entries
& $node .\tools\asar-tool.js grep $asar "measure_distance"
```

`tools/scan-strings.js` pulls printable runs out of a binary and filters them by regex — used to check whether the `.jsc` bundles carry recoverable strings.

Still worth trying

1. **The Ultra's firmware tarball.** Ask WeCreat support for the Ultra OTA file, unpack it, and run `strings | grep -E "^M[0-9]+"`. This is the cheapest possible route to a real M-code table.
2. **Runtime interception.** The renderer must decrypt `f.enc` to run. Anyone comfortable with Electron debugging could recover the decrypted bundle at runtime on their own machine.
3. `yuerugoutql/makeit-builder` — check whether it is public.
4. `opencv.js` is unencrypted; it may reveal how camera calibration is applied, which is relevant to 08-cameras.md.

12 — Start-of-Job Checklists

Why this exists

MakeIt has **Field-Lens Auto Detection**. LightBurn does not, and cannot — it will not switch device profiles because you physically swapped a lens.

That gap is not merely inconvenient on this machine. The Ultra has three lenses across two sources at two very different wavelengths:

- Wrong lens → wrong field size → job lands somewhere unintended, out of focus.
- Wrong source → **wrong goggles**. UV goggles (190–550 nm) give you nothing against 1064 nm fiber, and vice versa.

LightBurn supports a per-device **start-of-job checklist**: a free-text block stored in the device profile (`"Checklist"`, a top-level key in the `.lbdev`) that is shown before a job runs. It is the closest thing to a software interlock available to us, and it costs nothing.

Every profile in this repository ships with one. Please do not strip them.

The checklists

UV — Purple lens, 210 × 210 mm

```
LENS:     PURPLE field lens installed?          (210 x 210 mm)
SOURCE:   UV (355 nm) selected?
GOGGLES:  190-550 nm UV goggles ON?
FOCUS:    Focus set for this material height?
EXHAUST:  Extraction running? (UV on wood/leather/coated goods)
```

```
BED:      Correct lens for the job -- purple is FLAT and CYLINDRICAL work
```

UV — Green lens, K9 crystal, 70 × 70 mm

```
LENS:     GREEN field lens installed?          (70 x 70 mm ONLY)
SOURCE:   UV (355 nm) selected?
GOGGLES:  190-550 nm UV goggles ON?
FIELD:    Artwork fits within 70 x 70 mm?
NOTE:     True K9 internal 3D engraving is NOT available from LightBurn.
          This profile is for surface / 2D work through the green lens.
```

MOPA 60 W — Red lens, 210 × 210 mm

```
LENS:     RED field lens installed?            (210 x 210 mm)
SOURCE:   MOPA fiber selected -- NOT UV?
POWER:    This is the 60 W profile. Is this a 60 W machine?
GOGGLES:  800-1100 nm fiber goggles ON?  (UV goggles do NOT protect you)
REFLECT:  Workpiece secured? Specular metal reflects the beam
EXHAUST:  Extraction running?
```

MOPA 100 W — Red lens, 210 × 210 mm

```
LENS:     RED field lens installed?            (210 x 210 mm)
SOURCE:   MOPA fiber selected -- NOT UV?
POWER:    *** 100 WATT PROFILE ***  Is this a 100 W machine?
          A 60 W machine running 100 W settings will not behave as expected.
GOGGLES:  800-1100 nm fiber goggles ON?  (UV goggles do NOT protect you)
REFLECT:  Workpiece secured? Specular metal reflects the beam
POWER:    First run on this material? START LOW.
EXHAUST:  Extraction running?
```

Slide Extension

```
SLIDE:    Slide Extension mounted and clear of obstructions?
TRAVEL:   Full 520 mm travel path clear -- nothing in the way of the carriage?
LENS:     Correct field lens for the source in use?
GOGGLES:  Matching wavelength goggles ON?
NOTE:     LightBurn Split Marking is galvo-only and NOT available here.
          Slide positioning is manual or macro-driven. Verify before running.
```

Conveyor

```
BELT:     Belt clear, parts seated, load under 10 kg?
FEED:     Feed increment verified on a dry run WITHOUT the beam?
ATTEND:   Batch jobs run long. Do NOT leave unattended.
FIRE:     Extinguisher within reach? Extraction running?
LENS:     Correct field lens? GOGGLES: matching wavelength ON?
```

Editing them

Checklists live in `tools/profile-spec.json` under each profile's `checklist` key. `tools/build-profiles.ps1` writes them into the generated `.lbdev` files. Edit the spec, not the generated profile, so the two do not drift.

To change one in LightBurn directly: Devices → select device → Edit → the checklist field.

A note on tone

These read as shouty. That is deliberate. A checklist that people skim is worse than no checklist, and the failure mode being guarded against — wrong goggles on a 100 W fiber laser — is not recoverable.

13 — Setting Up a Test Machine

How to prepare the PC that has a Lumos Ultra physically attached, so it can run the bring-up stages and file results back into the repository.

The authoring machine and the test machine are usually different boxes — the laser tends to live in a workshop or basement, not next to your development workstation. This document assumes that split. If they are the same machine, skip to step 3.

Read SAFETY.md before the machine is powered on with a lens installed.

---

1. What the test machine needs

Requirement	Why	Required?
Windows 10/11	The tooling is PowerShell	yes (the scripts are Windows-first; ports welcome)
git	To clone the repo and commit captures	**yes**
LightBurn 2.1.02 or newer	2.1.00 added *"Allow for extra ok's from WeCreat firmware"*; 2.1.02 added a WeCreat device-detection fix and reverted the GRBL protocol to "GRBL-ish". Older builds desynchronise and produce the stuck-at-99 % / "laser busy" symptom	for Stage 3 onward
WeCreat MakeIt	Supplies the RNDIS USB driver, and is the reference implementation for Stage 5 captures	strongly recommended
Node.js	Only for asar-tool.js / scan-strings.js (MakeIt inspection)	optional
Python + pyserial	Only if you prefer miniterm for the Stage 1 serial probe. PuTTY works too	optional
Wireshark	Stage 5 differential capture on the RNDIS interface	optional, but it is the route to the M-code table

Nothing in Stages 0–2 needs LightBurn at all.

2. Get the repository onto the test machine

Pick whichever is least friction for you. All three end in the same place.

Route A — git bundle (no network account needed)

A git bundle is the whole repository, history included, in one file. Works over a shared folder, RDP clipboard, or a USB stick.

On the authoring machine:

cd "<repo>"

git bundle create lumos-ultra-lightburn.bundle --all

git bundle verify lumos-ultra-lightburn.bundle

Copy that file across, then on the test machine:

cd D:\DevHome *# or wherever you want it*

git clone lumos-ultra-lightburn.bundle "WeCreat Lumos Ultra Lightburn Config"

cd "WeCreat Lumos Ultra Lightburn Config"

git log --oneline *# confirm the history arrived*

To send later work back, bundle in the other direction:

git bundle create results.bundle --all

 # copy back, then on the authoring machine:

git pull <path-to>\results.bundle main

Route B — a private GitHub repo as the transport

Cleaner once you are iterating, because results flow both ways without copying files. A private repo is **not published** — you can flip it to public later and the history comes with it.

 # authoring machine

gh repo create wecreat-lumos-ultra-lightburn --private --source=. --remote=origin --push

 # test machine

gh auth login *# once*

gh repo clone <you>/wecreat-lumos-ultra-lightburn

Then the normal loop: test machine commits captures and pushes, authoring machine pulls.

Route C — plain file copy

If the test machine has no git, copy the working folder across and install git later. You lose history until then, so prefer A or B if you can.

Route D — one short path, for a machine with no shared clipboard

If you reach the test machine through a remote-desktop tool with no clipboard sharing (NoMachine, some VNC setups), typing long commands is miserable. Put a small launcher at the root of a share the test machine can read, and it needs one short path typed into **Win+R**:

```
@echo off
set "PS=%~dp0path\to\repo\tools\test-machine.ps1"
powershell -NoProfile -ExecutionPolicy Bypass -File "%PS%"
```

tools/test-machine.ps1 then syncs a local copy and offers the bring-up stages as a menu, so every stage is one keystroke. %~dp0 means the launcher works over any share name, drive letter or IP without editing.

3. Check readiness

With the **Ultra powered on and connected by USB**, and **MakeIt fully closed** (it holds the serial port):

```
cd "WeCreat Lumos Ultra Lightburn Config"
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\probe-workstation.ps1
```

This is read-only. It touches no hardware and fires nothing. It reports installed software, serial ports, RNDIS gadgets, every USB VID/PID, network adapters, LightBurn version, and whether git / node / python / gh are present — then prints a readiness summary.

Section 10 is the one to read. If it says *"No laser detected"*, work through:

- Is the machine powered on, not just plugged in?
- Is MakeIt closed? Check Task Manager for a stray WeCreat MakeIt! process.
- Is the RNDIS driver installed? It ships inside MakeIt at …\WeCreat\makeit-builder\resource\driver\win\rndis\ — run usb-driver-installer-x64.exe.
- Try a different USB cable and a directly-attached port rather than a hub.

4. Run Stage 0 and save the result

This is the single most valuable test in the project — it settles whether the Ultra is a GRBL serial device or a real galvo controller.

powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\probe-workstation.ps1 `

> .\captures\stage0-usb-<yourname>.txt

Then fill in captures/templates/usb-enumeration.md, save it alongside, and commit:

git add captures/

git commit -m "Stage 0: USB enumeration on <machine>"

What the result means is tabulated in 03-bringup-plan.md § Stage 0. The short version: Linux-gadget IDs plus a COM port confirms the current architecture; a BJJCZ/JCZ ID means the Ultra is a real galvo controller and much more of LightBurn becomes available.

5. Then Stage 1 and Stage 2

Stage 1 — serial identity. Still no beam. Talk to the port directly before involving LightBurn, so you see the raw truth rather than LightBurn's interpretation of it.

.\tools\stage1-serial-probe.ps1 *# auto-detects the CH340 port*

.\tools\stage1-serial-probe.ps1 -AllBauds *# if the default baud is silent*

Needs nothing but Windows PowerShell — no Python, no PuTTY. It leaves DTR and RTS alone so the controller is not reset, and sends only GRBL queries (?, $$, $I, $#, $G). **No M-codes** — WeCreat renumbers them between models. Template: captures/templates/serial-banner.md.

Stage 2 — the controller's HTTP API. Also no beam.

.\tools\stage2-rest-probe.ps1

Auto-detects the RNDIS subnet (confirmed as 192.168.42.0/24 on a real Ultra, controller at .1), scans the ports, and calls only read-only endpoints. It never calls /test/cmd/mcu — that one bypasses the lid interlock — and skips the head-moving autofocus probe unless you pass -AllowAutofocus.

6. Housekeeping on a test machine

- **Close MakeIt before every LightBurn session.** Most "cannot connect" reports trace to this.
- Keep the repository out of C:\Program Files and out of OneDrive-synced folders — both cause permission and locking surprises with git.
- If the test machine is headless or awkward to sit at, note that everything through Stage 2 is scripted and produces a text file you can collect later.

- Do not commit .pcap files or photographs; extract the relevant bytes into markdown instead. See captures/README.md.

14 — Driving MOPA Frequency and Pulse Width from LightBurn

Status: the codes are confirmed; emitting them from LightBurn is not yet tested. Read SAFETY.md first. Do not use this on real work until Stage 3 has verified that LightBurn's output reaches the machine intact.

The two codes

Both established by controlled differential capture of MakeIt's own G-code — one job, one parameter changed, everything else fixed. Each produced exactly one differing line. (evidence)

M38F<n> ; MOPA frequency

M39P<n> ; MOPA pulse width

M18S0 ; select the MOPA fiber source *(M18S1 selects UV)*

Values observed in the wild: F30, F48, F75, F151 · P200, P500.

Units are not established. Nanoseconds and kilohertz are the natural reading and fit typical JPT MOPA ranges, but nobody has verified the relationship between MakeIt's UI field and the emitted number. Establish that before building a material library on it.

Why this needs custom G-code at all

LightBurn *does* have Frequency and Q-Pulse Width fields — in its **galvo** device class. The Lumos Ultra presents to LightBurn as a **GRBL** device (why), whose cut settings are speed, power, passes, interval and the ordinary G-code set. No frequency field, no pulse-width field.

The workaround is that LightBurn's GRBL class supports **per-layer custom G-code**, which is the same granularity a material library needs anyway.

How to set it per layer

In LightBurn, for each cut layer:

Cut Settings Editor → Advanced → Layer G-Code

Put the parameters in that layer's start G-code:

M18S0

M38F75

M39P200

Every layer can carry different values. So a single job can run one pass at a short pulse for marking and another at a long pulse for depth, which is exactly the kind of thing a MOPA is for.

Suggested library structure

Rather than remembering numbers, build named layer presets:

Preset	M38F	M39P	Intended for
Brass — shallow mark			fill in from your own testing
Brass — deep relief			
Stainless — colour			colour marking is very pulse-width sensitive
Anodised — clean white			

Deliberately left blank. Publishing numbers we have not tested would be exactly the kind of confident guess this project refuses to make. Fill them in from your own material tests and contribute them back with the material, the source, and the resulting appearance.

Before trusting this

1. **Verify the units** — change MakeIt's field by a known amount and confirm the emitted value moves the same way.
2. **Find the accepted ranges.** The firmware may clamp, reject, or silently ignore out-of-range values. An ignored value is the dangerous case: the job runs at whatever was set last.
3. **Confirm LightBurn's output actually arrives.** Run a job from LightBurn with these codes, then read the resulting G-code back off the controller with tools/stage5-jobfile-grab.ps1 and check they survived. This is the whole point of Stage 3.
4. **Verify on scrap, at low power**, with the red lens fitted and 800–1100 nm goggles.

What this does not give you

Frequency and pulse width, and that is a great deal. It does **not** give you LightBurn's galvo-class features — 16-bit depth-map relief, 3D sliced images, lens correction, split marking. Those need the galvo device class itself, not just its parameters. See 09-k9-and-mopa-limits.md.

Evidence & Sources

Every non-obvious claim in this repository should be traceable to something here. If you add a claim, add its source. If you find a source that contradicts one of ours, open an issue — that is the most useful kind of contribution.

Retrieved **2026-08-24** unless noted.

Primary artifacts we inspected directly

Artifact	What it gave us
`WeCreat-Lumos-v1.5.lbdev` (WeCreat CDN, `?v=1754041991` → 2025-08-01)	`"Name":"GRBL"`, `"Type":"Serial"`, baud 1 000 000, 116 × 116 mm, `MirrorY`, `S_Scale`, `CutOrigin`, rotary keys, the Lumos start block, and the `M18`/`M1` macro labels
`WeCreat-Vision-V1.0.lbdev`, `WeCreat-Vision_Pro-V1.0.lbdev`, `WeCreat-Vista-V1.0.lbdev`, `WECREAT-LC.lbdev`	The Vision-family start/end blocks and the `M130`/`M21`/`M16`/`M17` macro labels; baud 500 000
`MakeIt!/resource/driver/win/rndis/rndis11.inf` (MakeIt 3.0.6, installed)	`USB\VID_0525&PID_a4a2` and `USB\VID_1d6b&PID_0104&MI_00` — the machine is a Linux USB gadget
`MakeIt!/resources/app.asar` (MakeIt 3.0.6, installed)	The nine `lumosUltraWorkMode/` assets; Electron; encrypted `f.enc`; bytenode `.jsc`
`MakeIt!/resources/app-update.yml` (MakeIt 3.0.6, installed)	Updater points at GitHub `yuerugoutql/makeit-builder`

Vendor `.lbdev` files are fetched by `tools/fetch-vendor-lbdev.ps1` into gitignored `reference/` and are **not redistributed** here.

WeCreat — official

- Lumos Ultra product page — https://wecreat.com/products/lumos-ultra-6w-uv-60w-100w-mopa-laser-engraver

- MOPA add-on module — https://wecreat.com/products/60-100w-mopa-laser-module-for-lumos-ultra
- Slide Extension — https://wecreat.com/products/slide-extension-for-lumos-ultra
- AutoFlow Conveyor — https://wecreat.com/products/auto-flow-conveyor
- Rotary Pro — https://wecreat.com/products/rotary-for-wecreat-lumos
- Software / LightBurn downloads — https://wecreat.com/pages/software
- **Instructions on Direct LightBurn Connection** ("Only USB connection is supported") — https://support.wecreat.com/hc/en-us/articles/9633347845519
- How do I Fix the USB Connection Problem? (RNDIS + CH340 in Device Manager) — https://support.wecreat.com/hc/en-us/articles/9775153916431
- How to Install USB Driver on the PC (`Resource > driver > win > rndis`) — https://support.wecreat.com/hc/en-us/articles/9081963351055
- Lumos Connection-Related Issue — https://support.wecreat.com/hc/en-us/articles/13440470222479
- Lumos Firmware-Related Issue (`.tar.gz` OTA) — https://support.wecreat.com/hc/en-us/articles/13489263547407
- Force OTA Guide — https://support.wecreat.com/hc/en-us/articles/9608509396367
- Software Version Information (MakeIt changelog, tops out at 3.0.2) — https://support.wecreat.com/hc/en-us/articles/9089646406159
- Tips for using LightBurn with WeCreat Lasers — https://support.wecreat.com/hc/en-us/articles/10560668639759
- Auto-pass Through Feeder with LightBurn (conveyor as **A axis**) — https://support.wecreat.com/hc/en-us/articles/10159360297231
- Tumbler engraving with LightBurn ("Set Axis to A") — https://support.wecreat.com/hc/en-us/articles/12836359966351
- Camera Calibration with LightBurn — https://support.wecreat.com/hc/en-us/articles/10662689424783
- Kickstarter campaign + FAQ — https://www.kickstarter.com/projects/wecreat/lumos-ultra-worlds-first-one-stop-uv-mopa-laser/faqs

LightBurn — official

- Devices window (import/export, `.lbdev` import-only in 2.x) — https://docs.lightburnsoftware.com/2.1/Reference/Devices/
- User Bundles (`.lbzip` contents) — https://docs.lightburnsoftware.com/2.1/Reference/UserBundles/
- Vendor Bundles (`.lbvendor`) — https://docs.lightburnsoftware.com/legacy/Vendor/VendorBundles
- Managing Preferences (`.lbprefs`) — https://docs.lightburnsoftware.com/2.1/Reference/ManagingPreferences/
- Machine Settings (`.lbset`) — https://docs.lightburnsoftware.com/2.1/Reference/MachineSettings/
- Add a Galvo Laser (JCZFiber / BSLFiber / EZCad3; "USB is currently the only supported type for Galvo devices") — https://docs.lightburnsoftware.com/2.1/GetStarted/AddGalvo/

- Device Settings — Galvo and Basic Settings (field, red dot, scale/bulge/skew/trapezoid, timings)
  — https://docs.lightburnsoftware.com/2.1/Reference/DeviceSettings/GalvoBasicSettings/
- Device Settings — Ports and Laser Settings (Laser Type CO2/Fiber/UV/SPI, Enable Q-Pulse Width, UV Min/Max Pulse)
  — https://docs.lightburnsoftware.com/2.1/Reference/DeviceSettings/GalvoPorts/
- Galvo-Specific Cut Settings (**Frequency**, **Q-Pulse Width**, 3D Sliced Image Mode)
  — https://docs.lightburnsoftware.com/2.1/Reference/CutSettingsEditor/GalvoSpecificCutSettings/
- Galvo Lens Calibration
  — https://docs.lightburnsoftware.com/2.1/Reference/CalibrateGalvoLens/
- Rotary Mode (Galvo)
  — https://docs.lightburnsoftware.com/2.1/Reference/RotaryMode/RotaryModeGalvo/
- Split Marking (2.1, galvo-only, external axis table)
  — https://docs.lightburnsoftware.com/2.1/Reference/SplitMarking/
- What's New in 2.1 — https://docs.lightburnsoftware.com/2.1/NewFeatures/NewIn2.1/
- Add A Camera / Camera Selection (WeCreat preset; RTSP not supported)
  — https://docs.lightburnsoftware.com/2.1/Reference/Cameras/AddACamera/ · https://docs.lightburnsoftware.com/2.1/Reference/Cameras/CameraSelection/
- **WeCreat Camera Calibration and Alignment** — https://docs.lightburnsoftware.com/2.1/Guides/WeCreatCameraAlignment/
- 1.7.00 release notes ("Initial WeCreat Vision camera support over RNDIS")
  — https://lightburnsoftware.com/blogs/news/lightburn-1-7-00
- 2.1 release notes ("Allow for extra ok's from WeCreat firmware")
  — https://lightburnsoftware.com/blogs/news/lightburn-2-1-quick-nest-enhanced-camera-support-undo-history-and-more
- 2.1.02 patch notes ("WeCreat device detection / messaging fix", "Reverted LightBurn GRBL Protocol to GRBL-ish") — https://lightburnsoftware.com/blogs/news/lightburn-2-1-02-patch-release

Reverse engineering

- `2ktsch/weburn` — WeCreat Vision → LightBurn bridge. REST endpoints, MD5 scheme, M-code constants, camera shim, and the "*THIS GCODE WILL RUN REGARDLESS OF THE LID POSITION*" warning — https://github.com/2ktsch/weburn
- weburn discussion thread — https://forum.lightburnsoftware.com/t/weburn-wecreat-vision-to-lightburn-compatibility-bridge/135403

Community — the load-bearing threads

- **Lumos is "some variant of a GRBL device"** (LightBurn staff), plus "we only discovered the Lumos after it had already launched" and WeCreat "didn't hear back"
  — https://forum.lightburnsoftware.com/t/possible-to-use-wecreat-lumos-rotary-combined-with-slide-extension-in-lightburn/179528

- **Depth maps do not work on the Lumos even with Pro** — "it uses GRBL... not a real galvo control laser and can't use depth map due that"
  — https://forum.lightburnsoftware.com/t/deep-engraving-wecreat-lumos/181803
- **Serial banner** `[WeCreat Lumos :ver 000207]` with extra `ok`s
  — https://forum.lightburnsoftware.com/t/wecreat-lumos-busy-and-not-finding-my-laser/179777
- `M130X…Y…` **autofocus**, and the Focus Z button only existing on Custom GCode + GRBL devices — https://forum.lightburnsoftware.com/t/focus-z-button-for-wecreat-shown-in-the-wecreat-camera-calibration-video/153627
- WeCreat + LightBurn starter guide (community)
  — https://forum.lightburnsoftware.com/t/wecreat-using-lightburn-starter-guide/138723
- `M15S1`/`M15S0` **= exhaust fan;** `M130X50Y50` **= autofocus** — https://thetinkerverse.com/lets-talk-about-the-wecreat-vision-40w/
- Lumos camera works in LightBurn (rotated 90°)
  — https://forum.lightburnsoftware.com/t/lumos-camera-setup/186137
- Vision passthrough conveyor as A axis — https://forum.lightburnsoftware.com/t/wecreat-vision-with-passthrough-conveyor-a-axis-not-working/158871
- **LightBurn cannot switch Lumos laser sources** — https://grouchyfarmer.com/2025/12/27/wecreat-lumos-dual-laser-engraver-follow-up-questions-etc/
- AlgoLaser `M16`/`M17` = air pump — **contrast case, do not import** — https://forum.lightburnsoftware.com/t/algolaser-air-assist-how-to-use-m16-m17-effectively/127992

Press & reviews (used only for specs, never for protocol)

- PR Newswire launch — https://www.prnewswire.com/news-releases/wecreat-launches-lumos-ultra-on-kickstarter-the-worlds-first--only-one-stop-6w-uv--60w100w-mopa-laser-system-302709013.html
- Hackster — https://www.hackster.io/news/wecreat-s-lumos-ultra-delivers-multi-material-high-speed-etching-with-dual-uv-and-mopa-lasers-95c89af78fc1
- HobbyLaserCutters review (lens focal lengths; 3D modes are MakeIt-only)
  — https://hobbylasercutters.com/wecreat-lumos-ultra-review/
- GizmoCrowd — https://www.gizmocrowd.com/post/wecreat-lumos-ultra-laser-engraver-review
- MakeTechCreate — https://maketechcreate.com/blogs/video-tutorials/wecreat-lumos-ultra-review-uv-mopa-laser-engraver-tested-on-wood-metal-glass-more
- Tom's Hardware, WeCreat Lumos (116 × 116 mm field)
  — https://www.tomshardware.com/maker-stem/wecreat-lumos-review

---

Known gaps in our own research

Stated so nobody assumes these were checked:

- **Reddit** was not searched (r/lasercutting, r/fiberlasers). Likely place for an early Ultra owner report.
- **YouTube** descriptions, comments and transcripts of Ultra review videos were not read.
- **GitHub code search** was unavailable — no search for `M18S1`, `M57A`, `M107X`, `M130X` in a WeCreat context, and `weburn`'s forks and issues were not enumerated.
- **FCC ID filings** were not retrieved. **FCC internal photos are the best public route to a definitive answer on the controller board** and should be someone's next move.
- **Facebook groups and Discord** are not indexed.
- The **LightBurn forum's own search** is robots-disallowed; we enumerated the WeCreat category but cannot rule out an Ultra thread elsewhere.
- Whether `yuerugoutql/makeit-builder` on GitHub is public.

Any of these could be closed by a contributor in an afternoon.

Branding assets

Original artwork for this project, created by Lee Gillie.

File	Use
LightburnConfigProject.png	Wide banner. Used at the top of the repository README
LightburnConfigIcon.png	Square icon. Intended for GitHub's **social preview** (Settings → General → Social preview) and as an organisation/avatar image

Two things to keep in mind

The banner is aspirational. It says "Tested & Tuned by the Community", which describes where this project is going rather than where it is. The README carries a caption saying so, and points readers at the feature inventory for real status. If the artwork is ever re-rendered, that phrasing is worth revisiting.

The lens part numbers shown on the banner are decorative, not photographs of real hardware. They are not evidence and must not be cited as such. Real, verified hardware details live in captures/.

Third-party marks

The banner includes the LightBurn, WeCreat and GitHub logos, with a trademark disclaimer in the lower-left. Those marks belong to their respective owners and are used here only to identify the equipment and software this project works with. This project is not affiliated with, endorsed by, or supported by any of them.

profiles/

Device class: Custom GCode, flavor `wecreat`

On first connection LightBurn 2.1.04 warned that a WeCreat machine had been detected but the device was not "a custom gcode device with the WeCreat device flavor". These profiles have been **regenerated accordingly**, using the exact key set LightBurn's own wizard writes:

```
"Name": "Custom GCode",
"Settings": { "GCodeFlavor": "wecreat", "BaudRate": 1000000, "S_Scale": 1000, ... }
```

Note that WeCreat's own `WeCreat-Lumos-v1.5.lbdev` declares `"Name": "GRBL"` — it dates from August 2025, predates the flavor, and now produces that same warning. **The vendor's published profile is outdated.** See the capture.

The Custom GCode class has a much smaller settings block than GRBL: no `CutOrigin`, no `rotary*` keys, no `Sim_*`. Anything from a GRBL profile that is not in the list above is simply not read.

`draft/` — generated, unverified

Six LightBurn device profiles for the Lumos Ultra. **None has been tested on a physical machine.**

File	Source	Lens	Workspace
`WeCreat-Lumos-Ultra-UV-Purple-210.lbdev`	6 W UV	Purple	210 × 210 mm
`WeCreat-Lumos-Ultra-UV-Green-K9-70.lbdev`	6 W UV	Green	70 × 70 mm
`WeCreat-Lumos-Ultra-MOPA60-Red-210.lbdev`	60 W MOPA	Red	210 × 210 mm
`WeCreat-Lumos-Ultra-MOPA100-Red-210.lbdev`	100 W MOPA	Red	210 × 210 mm
`WeCreat-Lumos-Ultra-Slide-520x183.lbdev`	any	any	520 × 183 mm — **speculative**

File	Source	Lens	Workspace
`WeCreat-Lumos-Ultra-Conveyor-206.lbdev`	any	any	206 × 206 mm — **speculative**

What is deliberately missing

Every profile ships with:

- **empty `StartGCode` and `EndGCode`**
- **no macros**
- `EnableZ: false`

That is not an oversight. WeCreat renumbers M-codes between machine models, and **no Lumos Ultra M-code has been verified by anyone**. Candidate start blocks and macros are documented in ../docs/04-mcode-dictionary.md and stay there until hardware confirms them. See ../SAFETY.md.

Each profile does carry a **start-of-job checklist** — LightBurn's only substitute for the machine's field-lens auto-detection, which LightBurn cannot use. Please don't strip them.

Installing one

LightBurn → Laser window → **Devices** → **Import** → pick the `.lbdev`.

Then work through ../docs/03-bringup-plan.md from Stage 3. Do not run a job before Stage 4.

Regenerating

Profiles are **generated**, not hand-written. Edit the spec, not the output:

```
# edit tools\profile-spec.json, then
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\build-profiles.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\validate-lbdev.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\tools\summarize-lbdev.ps1
```

`validate-lbdev.ps1` enforces the hygiene and safety rules above and runs in CI. A PR that ships a profile with non-empty start G-code will fail the build.

`bundle/` — not yet

Once profiles are confirmed on hardware, this will hold a single `WeCreat-Lumos-Ultra.lbzip` User Bundle carrying all profiles plus per-source material libraries, so users import once instead of six times. LightBurn: `File → Bundles → Import Bundle`, or drag the `.lbzip` into the window.